A project report on

HOSPITAL MANAGEMENT SYSTEM USING MERN STACK

Submitted in partial fulfillment for the award of the degree of

M.Tech [Integrated]

Computer Science and Engineering

by

B SAHITHYA (20MIC0007)

Under the Guidance of

Dr. Manoov R

Assistant Professor Sr. Grade 1

School of Computer Science and Engineering (SCOPE)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

April, 2025

HOSPITAL MANAGEMENT SYSTEM USING MERN STACK

Submitted in partial fulfillment for the award of the degree of

M.Tech [Integrated]

Computer Science and Engineering

by

B SAHITHYA (20MIC0007)

Under the Guidance of

Dr. Manoov R

Assistant Professor Sr. Grade 1

School of Computer Science and Engineering (SCOPE)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

April, 2025

DECLARATION

I here by declare that the thesis entitled “HOSPITAL MANAGEMENT SYSTEM USING MERN STACK” submitted by me, for the award of the degree of M.Tech [Integrated] Computer Science and Engineering is a record of bonafide work carried out by me under the supervision of Dr. Manoov R.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 16-04-2025

Signature of the Candidate

CERTIFICATE

This is to certify that the thesis entitled “HOSPITAL MANAGEMENT SYSTEM USING MERN STACK” submitted by B SAHITHYA (20MIC0007), School of Computer Science and Engineering, Vellore Institute of Technology, Vellore for the award of the degree M.Tech [Integrated] Computer Science and Engineering is a record of bonafide work carried out by him/her under my supervision.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The Project report fulfils the requirements and regulations of VELLORE INSTITUTE OF TECHNOLOGY, VELLORE and in my opinion meets the necessary standards for submission.

Signature of the Guide **Signature of the HoD**

Internal Examiner **External Examiner**

ABSTRACT

In recent times, the software applications provide the services faster than traditional services. The appointment in the hospital takes a long time and you have to wait for your turn to visit the doctor. It is such a long and time taking process. Instead, we can do the same thing through the latest software technologies to maintain appointments, prescriptions and medicines. The main of this project is to provide efficient approach to reduce waiting time, easy schedules, payments, and doctor visits. By leveraging MERN Stack technologies, which provides both front end and back end services, we intend to create web applications for both doctors and patients for scheduling appointments.

Our solution provides the three key players - Admin, Doctor, and Patient, with an easy-to-use user interface (UI) for the purpose of improving the medical sector. The technology streamlines several administrative as well as medical processes by putting multiple functions under one umbrella, ensuring better patient care and efficient hospital operations. The Admin role plays a vital part in ensuring the operation of the system's working backbone. Admins are capable of controlling physician schedules by distributing available time and making it sure that patients should be able to book appointments by considering the accessibility of medical experts. Further, admins take responsibility for handling stocks of medicine in such a manner that they always have enough supply and medicine stocks are present if prescribed by doctors.

The application enables doctors to schedule appointments, prescribe medicines, and interact with patients according to the schedule. Doctors can organize the appointments by selecting their free time in the application to avoid clashes and view the schedules in real time. Doctors can prescribe the medicines through the platform and patients can view them in their login. The history of prescriptions is also present for future use. They also manage the dispensing of medications, ensuring that patients receive the dosages recommended. Prescriptions are present in the application any time which improves the medical records storage and reduces paperwork and also eliminate manual errors.

Patients are able to interact with the doctor through the patient login and book the appointment according to the doctor schedule. Depending on the admin-allotted time schedules in their login, patients can access the application and book the doctor of their choice and specialization. By eliminating the need for manual booking of appointments, this self-service facility streamlines and accelerates the process. In pursuit of enhancing convenience, patients may also view their prescription history to ensure that their medical records remain accessible at any given time. HMS is a MERN Stack application.

ACKNOWLEDGEMENT

The project “Hospital Management System Using MERN Stack” was made possible because of inestimable inputs from everyone involved, directly or indirectly. First, I would like to thank and express my sincere gratitude to my guide, Prof. Dr. Manoov R, who was highly instrumental in providing an innovative base with constructive inputs for the completion of the project

I would like to express my gratitude to DR. G VISWANATHAN, Chancellor VELLORE INSTITUTE OF TECHNOLOGY, VELLORE, MR. SANKAR VISWANATHAN, DR. SEKAR VISWANATHAN, DR. G V SELVAM, Vice – Presidents VELLORE INSTITUTE OF TECHNOLOGY, VELLORE, Dr. V S Kanchana Bhaaskaran, Vice – Chancellor, Dr. Partha Sharathi Mallick, Pro-Vice Chancellor and SCOPE Dean Incharge Dr. N. Jaisankar for all the support provided at the school for successful completion of the project.

I would also like to acknowledge the role of HOD of the Dept. of Computational Intelligence Dr. Swathi J N, who was instrumental in keeping me, updated with all necessary formalities and helped me in all aspects for the successful completion of the project.

Finally, I would like to thank Vellore Institute of Technology, for providing me with a flexible choice and for supporting my project execution in a smooth manner.

Place: Vellore

Date: 16-04-2025

Name of the Student

B Sahithya

CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Expansion
MERN	MongoDB Express.js React.js Node.js
HMS	Hospital Management System
UI	User Interface
JS	Java Script
MDP	Markov Decision Process
SQL	Structured Query Language
PHP	Hypertext Pre Processor
CRUD	Create Read Update and Delete
UML	Unified Modeling Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
DFD	Data Flow Diagram

1. INTRODUCTION

1.1 THEORETICAL BACKGROUND

Technology in the health sector for improved patient care and more effective operations has grown tremendously. There used to be much manual activity involved in traditional hospital administration, such as appointment scheduling over the phone, records in paper form, and manually monitoring inventory. All these approaches had a lot of inefficiencies in the form of lost paper work, inaccurate appointment scheduling, and challenges in managing the humongous amount of medical data. It was needed to make these processes more advanced by adopting computer technologies as the healthcare services rose.

To address these challenges, a Hospital Management System (HMS) was created. Most of the key hospital operations like prescription management, inventory management, medical record management, doctor scheduling, and patient registration are computerized by HMS software. As web-based applications have become very popular, HMS platforms are now accessible from any location and give real-time information to patients, physicians, and hospital administration. This has enhanced overall patient care, minimized human error, and enhanced operational efficiency.

1.2 MOTIVATION

The key motivations of the projects include:

- **To enhance operational efficiency:** Hospital staff can focus on patient care through minimizing administrative responsibilities through automation of processes such as inventory management, prescription processing, and scheduling appointments.
- **For enhancing patient experience:** Convenience and satisfaction are enhanced when patients are provided with easy access to appointments, drugs, and medical records through a user-friendly interface.

- **To access real-time data:** Accessing real-time data by doctors, patients, and administrators is made possible to ensure better coordination and faster decision-making.

- **To reduce human errors:** Documenting and procedures digitization reduces the risk of errors, enhancing accuracy in areas such as patient information and stock control.

- **To provide flexibility and scalability:** The MERN stack allows for the system to be easily expanded in order to accommodate increasing demands of the hospitals so that it can adapt to evolving healthcare scenarios.

1.3 AIM OF THE PROPOSED WORK

To design an exclusive user friendly platform for both doctors and patients to perform administrative, medical, doctor-patient related interactions such as appointment scheduling and generating prescriptions for patients using MERN stack technology - MongoDB, Express JS, React JS, Node JS. To integrate flexibility and scalability by creating simple platform which can help users to understand our application better.

1.4 OBJECTIVE(S) OF THE PROPOSED WORK

- To implement the MERN stack in order to make the application scalable and flexible so it can easily include new features and extend the system in the future.

- To enhance efficiency in administration through the provision of resources for tracking inventory, scheduling appointments, checking doctor availability, and generating hospital prescriptions.

- To enhance patient care through offering an easy user-friendly interface to the patients for booking appointments, viewing prescriptions, and accessing medical information.

- To allow real-time data access by doctors, admins, and patients to speed up access to the site.

1.5 REPORT ORGANIZATION

Introduction: This part introduces the project and its significance. It explains why a hospital management system helps to organize the appointments and ease with patient's problems.

Literature Survey: This section gives the summary of previous works of the papers and helps the project to refine better by pointing out the disadvantages. I proposed a problem statement which has important key to implement our proposed methodology.

Methodology: This section gives an overview of the technologies used and how the project is implemented. I described the technologies for both frontend and backend. It also gives information about these tools can improve the UI of hospital management system.

Results: This part shares what happened when I ran the code from both frontend and backend. We talk about how the user interface is easy for the patients to use and use this platform to book the appointment. The results show how admin, patient and doctor profiles work respectively.

Discussion: Here, I interpreted the results and screenshots of the website created. I discussed about the challenges I faced during the project and how I overcame them and how this project is helpful for both doctors and patients.

Conclusion: This section summarizes what I found and why it matters. I mentioned about any future work that could make the program even better and ended with the final thought about why this project is important.

References: Lastly, I listed the research papers from Google scholar which were helpful for my project to perform literature survey.

2. LITERATURE SURVEY

2.1 SURVEY OF THE EXISTING MODELS/WORK

[1] Smart software applications can manage medical patient appointments in a very effective way. With the use of smart technologies, UAE developed a website that contains mobile app, website and their own database to store the patient's information. . It integrates technologies like C#, MySql to complete the website to provide appointments to the patients. The proposed methodology Embedded Expressed Shell(EES) and AI to solve the diagnostic problems using human intelligence and also the doctor's advice to solve the patient's health problems. EES was integrated into the application by using Visual Studio with C# to create the inference engine. This proposed system has been verified using different test cases to check how it can improve further. User logins to website and sends a query and the system answers to the query according to the data available. Overall, the website is successfully giving 75% current suggestions to the users and result of the process is called as Inference Engine (IE).

[2] The Mobile Medical System is designed especially for families to take care of their health in an organized way. The application is designed to store records of patients such as appointments and prescriptions given by doctor. The proposed methodology uses MERN Stack technologies where MERN represents MongoDB, Express JS, React JS, and Node JS. MERN is a full stack application which has both front end and back end. The application is user friendly, which keeps track of remainders, appointment and prescriptions. Along with MERN Stack, Raspberry Pi 4 is used for the sake of deployment. It is a minicomputer to build our application and also helps to access the information. Overall, the application is user friendly, scalable and robust. The main goal of the paper is to create a application to store prescriptions of the patients so that user can have clear overview of their health.

[3] This paper discusses the development of websites and web applications to manage the booking medical appointments. The proposed methodology is SCRUM and Django framework. This framework allows the developers to program in python language and has MVC. SCRUM is an iterative development process model. The work of the SCRUM process is divided into 4 subparts and each team has to do the allotted task in certain period of time. MVC handles business logic, user interaction and shows the data to the user. The application has CRUD operations in the admin side login. Overall, the final website is user friendly, gives fast response and manages the integrity of the application.

[4] This paper discusses the improvement of medical system and implementation of an expert system for hospital appointment scheduling. It employs the technologies of internet based expert systems such as MYCIN, PUFF, CASNET, etc to deliver the application early. These expert systems improve the doctor patient interactions, and reduce the waiting time, and saves time for both doctor and patient. The patients can book an appointment with the help of internet and the ones who don't the exact department to choose are directed to embedded systems in the application. The decision trees are made to categorize the patient into correct department. SQL and ASP.NET is used for application development and queries. Overall, the internet based applications has potential advantages in healthcare.

[5] This literature paper gives an overview of accurate approach to improve the vaccination appointment system in vaccination centers. It implements the integration of database technologies with mobile computing to improve the quality of service for patients, medical staff and doctors. The paper gives the overview of how to manage vaccination process for children and to keep track of the vaccination and other medical records. The study also proposed an Android-based mobile application and a web application to book an appointment and to maintain and manage health records of the patients. The application is designed to give importance for both patients and medical staff, and its goal is to reduce the patient waiting time and improve healthcare. The paper also gives the overview of the proposed architecture, where the architecture highlights different layers used such as Resource, Service and Presentation layers. Also, they

mentioned related works in healthcare system and it's used and is grateful to the Palestine Technical University for publishing their research.

[6] This paper mentions the disadvantages of booking medical appointments for nuclear medicine examinations in a large Chinese hospital. The paper addresses the problems related to radiopharmaceuticals and the behaviour of the patients. The authors of the paper propose a methodology to schedule online appointments using the following to reduce the disadvantages. The schedules in the application are categorized as FCFS, Offopt, Offline model and dynamic model. Dynamic is considered due to threshold level. They also conducted various test to show the effectiveness of the proposed methodology, showing the major improvements in result examination, resource utilization, and timely rate. The paper also explains how other algorithms like genetic algorithms and the calculation of sensitivity analysis can improve the booking of appointments in nuclear examinations.

[7] This paper explains the disadvantages of scheduling the patient appointments within a moving booking window. To address this, they introduced a Markov Decision Process (MDP) model to maximize the reward and also reduces the capacity cost. Along MDP, they also used FCFS, STP, MP, and ASP for comparison purposes. The best practices are defined, as are structural aspects of the optimal advance scheduling strategy, such as the multimodularity and supermodularity. It proposes efficient policies based on the results from conducted tests to compare the efficiency and effectiveness of the MDP. As for rewards, multiple booking thresholds are preferred. The two policies assignment sequence policy and multiple threshold policy are proposed for simulation. This paper also examines the advantages of the proposed methodology to improve the schedules in moving booking window.

[8] This paper gives an overview of the impact of creating an medical booking system for outpatient clinics. The designed application was user friendly and secure. The proposed methodology includes various technologies such as PHP, MySql. To check the status of the application, the evaluation metrics compares the results before and after the implementation in ten clinics. The evaluation metrics used are patient waiting time, doctor available time, service time, no show rate, etc. The analysis made 95% of

confidence level which concludes that there is less waiting time for patients and increased doctor availability in the clinic. Hence, the online appointment scheduling algorithm was successful in implementing various performance metrics and improved medical care for patients. Due to the successful implementation, they created app for both iOS and android versions.

[9] This literature paper identifies the challenges of scheduling medical appointments and proposes a model that increases efficiency in booking the appointments to decrease no show which interrupts the healthcare system. This paper uses machine learning models to calculate the patient attendance. Decision Tree, Neural Network, Support Vector Machine, Linear Regression, Naive Bayes are used to perform predictions. The success rate of proposed models is calculated by various performance metrics such as accuracy, precision, recall, F1 score. The application was created using AMPL language and solved using CPLEX solver. Overall, this paper discusses the use of machine learning algorithms to predict patient attendance and improves the challenge of appointment scheduling by creating time slots with overbooking strategy.

[10] This literature paper mainly focuses on optimizing the medical appointment scheduling for outpatients in a diagnostic centers, particularly for Single Photon Emission Computed Tomography (SPECT) scanning purposes. It explains the radiopharmaceutical compound that can be delivered to the clinics in a very short period of time which can minimize violations and maximize the resource availability. The study introduces a dynamic robust optimization framework to handle these challenges and validates the model using data from a molecular imaging center in Tehran, Iran. It reviews important studies related to radiopharmaceutical administration and appointment scheduling in medical imaging centers. The paper also addresses the details of molecular imaging procedure during the operation, stating the connection of components involved in the imaging process.

2.2 GAPS IDENTIFIED IN THE SURVEY

- The application of this paper is available only on iOS which has locked out many non-Apple users. Also, the average cost of an iOS is very high and not everyone can afford it [2].
- There is no mention of doctor assigning prescription and medicines to the patient. Inventory and prescription are not handled by the admin [3].
- The Mobile Application has logins for different profiles but does not support CRUD operations. CRUD operations are very important for the admin to maintain the details of the user [5].
- All the papers did not clearly explain about admin functionalities in the website. For example, Admin should be able to add users of all types using add user button – doctor, and patient. The role of each user is not mentioned in the total users' page.

2.3 PROBLEM STATEMENT

Manual inefficiencies that the hospitals routinely adopt to handle appointments, medications, and inventories result in errors, delay time, and hindrances to business. Physicians don't have handy tools to keep track of schedules and medications, and patients cannot schedule appointments and see medical records. Hospital management is busy doing manual tracking of inventory and schedules of physicians and hence doesn't utilize resources properly. These problems adversely affect the quality of patient care and operational efficiency. Thus, there is a requirement for an automated, centralized, and user-friendly system to solve these problems. Thus, to operate hospitals efficiently is a feature-rich, web-based Hospital Management System (HMS) based on cutting-edge software technologies like MERN Stack, which help in automating hospital's key functions, makes working more efficient, ensures better and secure patient care, and scalability.

3. OVERVIEW OF THE PROPOSED SYSTEM

The main technologies which are used for this project is MERN Stack. Let's get the brief overview of the frontend, backend and the databse used to implement the application. The following tools and technologies have been used:

3.1 Front End

There are 3 different logins – Admin, Doctor and Patient. React JS project is classified into client and server. React is used for client. To install react project, run the command "npx create-react-app ." in the VS code. The node modules are installed and we are ready to perform frontend logic. The other packages required for the implementation can be installed accordingly by "npm install" command.

3.2 Back End

Node JS is installed in the VS Code by using npm install or directly installing latest version from the Google. Express JS is a Node JS framework used to build the application offering middleware, routing, etc. The backend is divided into 4 parts mainly. They are - Controllers, models, middlewares, and routes.

3.3 Database

For database, I used MongoDB. Create a MongoDB account using and select the dropdown menu to create a new project. It ensures to store the user credentials and their information.

3.4 Dotenv and Axios

Dotenv is used to manage all our sensitive configuration variables, such as database connection strings, API keys, port number and secret tokens. Axios is a node package which is used to fetch the doctor prescription, appointments, and their respective information from the database. And axios is also used in user authentication.

4. REQUIREMENT ANALYSIS AND DESIGN

4.1 REQUIREMENT ANALYSIS

Requirement gathering was conducted through brain storming, interviews, research papers analysis. The main objective of the requirement gathering is to understand the problems of the users and to decide which technology can be adopted to perform the implementation.

- **Brainstorming:** Brainstorming has let me in many ideas for starting a project. To question myself which technologies are better for implementation. Problem statement was framed with the help of interviews, research paper analysis.
- **Interviews:** Conducting interviews of general audience to know what kind of opinions they have regarding this project statement. They also shared their pain points and how they wanted specific kind of UI to ease the streaming of the website.
- **Research papers:** Research papers are collected through Google scholar website. All the quality research papers are separated and are used for research purposes. About 10 papers are collected and I found the gaps in the literature survey and framed a problem statement.

The result of the requirement gathering led to functional and non functional requirements which are listed in the upcoming sub sections. These requirements are foundation of the proposed system.

4.1.1 Functional Requirements

The first and foremost requirements to move forward for the implementation are functional requirements. The functional requirements give the idea of the application to be created and what are important functionalities to be implemented to make the website to work user friendly.

- **Authentication and Authorization:** The webpage allows admin, doctor and patient to have separate logins securely. All the credentials are stored in mongo db database and can control the role based access for every user. There are separate restrictions for each user in their logins.

- **Appointment Management:** Patients can choose the doctor of their choice and book appointment in the available slots. Doctor shall add their specifications, available timings and book the slot for interactions. Admin has the authority to add, delete and update details of both doctor and patient and manage the schedule accordingly.

- **Prescriptions:** After checking the patient, doctors write the prescriptions and them in the prescriptions tab which can be viewed by patient. Patient can also view the prescription history whenever they want. Admin can make records of the history of prescriptions.

- **Patient Records:** The application has the history of medical records, appointments and user profiles. The authorized users (admins) can access the information and update patient and doctor information accordingly.

- **Search feature:** The application is designed so that patients can select the doctor of their choice and filter the specializations and schedule the appointment.

4.1.1.1 Product Perspective

The Hospital Management System is web application that implemented using MERN Stack - MongoDB, Express JS, React JS, Node JS. This web application is built to ease the physical process and manual paper based work for scheduling appointments, receiving and adding prescriptions and to store patient's data. This is a centralized application with Mongo DB acting as the database to authorize the admin, patient and doctor and ensure a smooth access of the website.

4.1.1.2 Product Features

The product features include:

- Secure and safe user signups/registration according to their respective role - Admin, Doctor and Patient.
- Booking the appointment and managing the schedule.
- Select the doctor with respect to their specialization.
- Doctor giving the prescriptions and having the history.
- Admin keeping the track of doctors and patients.
- Search filter and allowing user to search doctors.
- Select the available slots of the doctor.
- Admin can perform CRUD operations.

4.1.1.3 User Characteristics

There are 3 users of hospital management system – Admin, Doctor and patient. The roles and features of these actors are:

- **Admin:** Admin has different dashboard that contains information of all the users. Responsibilities of admin include adding, deleting, updating the user details and also doctor scheduling, tracking appointments and having track of prescriptions.
- **Doctor:** Doctor's dashboard allows them to add prescriptions, add their available slots and access the patient data. Doctor should add their specialization so that patient can select the doctor of their need.

- **Patient:** Users who need the application to have an appointment with doctor and their pain points. User can book the doctor according to their problem and book an appointment. They can access their prescription which is given by the doctor.

4.1.1.4 Assumption and Dependencies

4.1.1.4.1 Assumptions

- To access the webpage, user need to have stable internet connection and any web browser.
- The information of the users - Admin, Doctor and Patient are checked.
- The patient data, doctor data and prescription history are stored in the database.
- The system is properly deployed and maintained properly up to date.
- Doctors and patients should have minimal knowledge of how to access the website and use their dashboard respectively.

4.1.1.4.2 Dependencies

Software: Visual Studio Code, Install Node JS, install React JS and node modules, Express JS and Mongo DB cluster, .env file to connect mongo db, HTML, CSS, Bootstrap and GitHub repository to push to the code. Any kind of deployment method to deploy and use the website irrespective of local host.

Hardware: A computer with specific memory and stable internet connection. Window 10 or above.

4.1.1.5 Domain Requirements

- The application must have the best healthcare norms to make sure the data in webpage is safe and secure.
- Only registered medical professionals - doctors can prescribe the medicines to the patients. Doctor should add their specializations to ensure smooth appointments.
- Appointment schedule should not clash with other slots. Real time monitoring to check if once slots are booked they cannot be repeated to avoid double booking.
- User accesses are monitored and admin can view the history of patient and doctor and has control over the website.
- Adding and deleting of the user according to requirements and make to check if they added/deleted properly.
- The visual studio code should handle the frontend and backend code. Install Node JS to create react application.
- The code should install all the required react packages whenever needed to make the user interface better.

4.1.2 Non-Functional Requirements

- **Performance:**

  The response time of the web application when a user request is sent should be around 2 seconds. The webpage should be able to handle 100 users without performance at the least.

- **Scalability:**

  The web application must be able to handle the increasing users, appointments and user response if the number of the users rapidly increase. The webpage should integrated billing - if implemented in the future.

- **Security:**

  The important information regarding user details should be protected. The database must allow access to admin so that they can ensure safety of user personal information. The application should also resist to common attacks and vulnerabilities.

- **Reliability and Availability:**

  The application should be available to the user at any time - 24/7 with 99% uptime. The user information and features of the website should have backup in case of any recovery to prevent the data loss.

- **Usability:**

  The user interface should be designed in a way so that it is understandable to all users even if they minimal experience with web pages. It should have seamless experience in all devices - mobiles, tablets, laptops.

- **Maintainability:**

  The application should follow the coding practices to ensure that it can allow future changes. The design should be made according to manage the changes easily. All these future changes can be documented for future.

- **Portability:**

  The system should be deployed to be used globally irrespective of the device in which webpage is created. The application should be deployed for both frontend and backend to ensure responses are generated for the user requests. The deployed link should be browser independent and platform independent.

4.1.3 System Modeling - DFD for each Module:

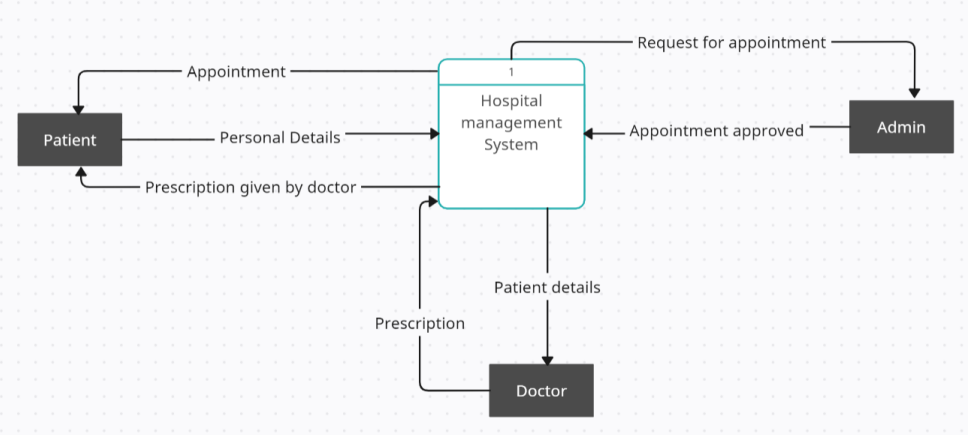

Fig 4.1 DFD Level 0

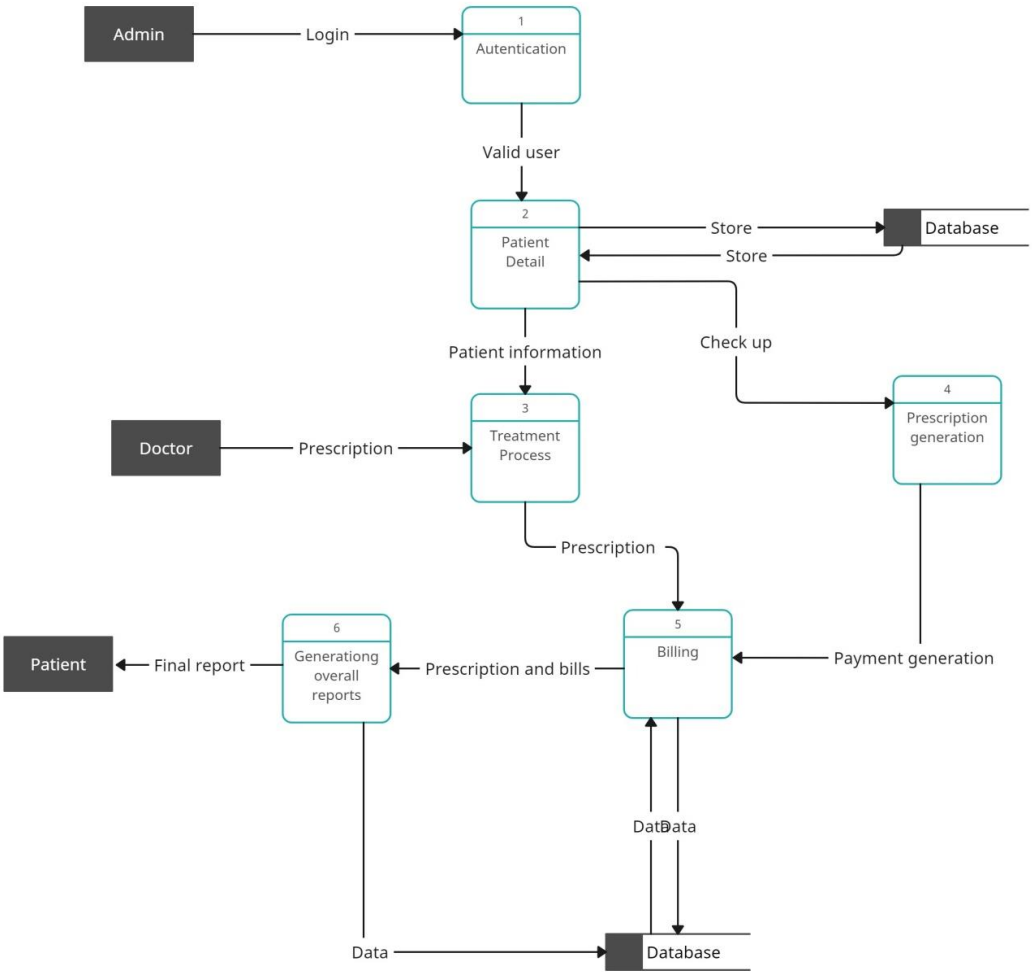

Fig 4.2 DFD Level 1

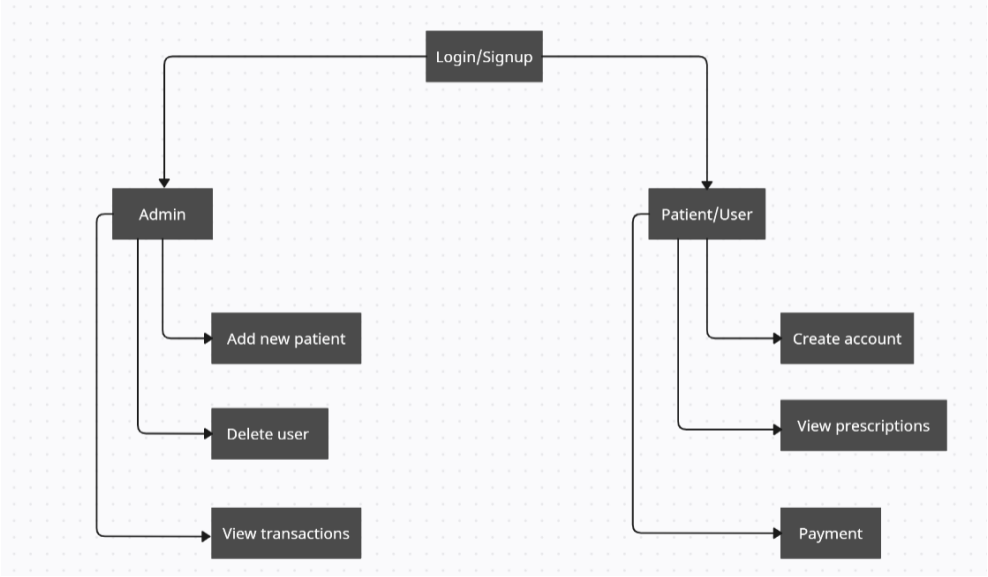

Fig 4.3 DFD Level 2 for login

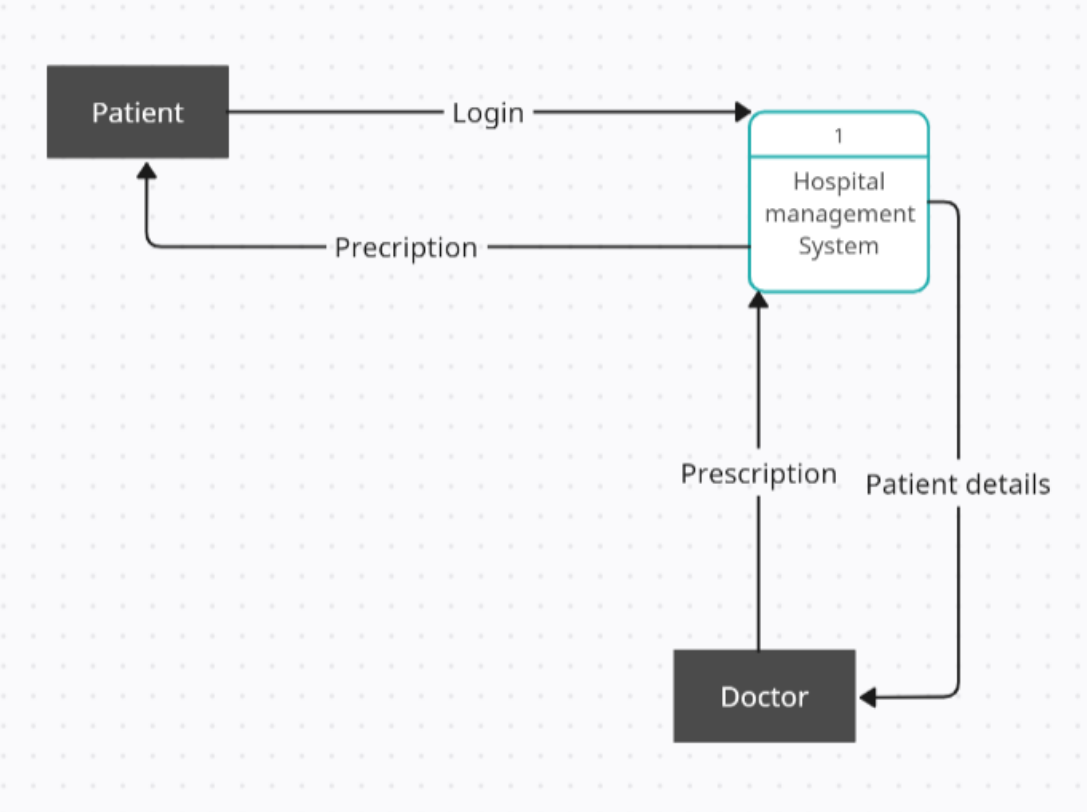

Fig 4.4 DFD Level 2 for doctor giving prescription

4.1.4 Engineering Standard Requirements

The Hospital Management System is designed by various technologies and engineering standards to ensure that this application meets the expectations of wide range of audience. The main aim is to provide a secure, reliable and user centric platform for the users to support healthcare facilities while they are using application in any location. The application determines performance, legality, ethical responsibility and sustainability to make sure that webpage is suitable for deployment. it should be suitable for integrating the features for both small clinics and large scale hospitals. By incorporating engineering standard requirements, we can ensure software engineering, data privacy, scalability and usability. The application allows a smooth workflow for all the logins which can improve patient experience so that they can recommend our platform to others. It also supports long term technical enhancements to transform healthcare facilities digitally available for any future update.The expectations regarding performance, responsibility and reliability across various domains explained below:

4.1.4.1 Economic Requirements:

- The Hospital Management System using MERN Stack reduces the physical process and manual work with involves lot of human errors and difficulty in reaching the doctor.
- It makes the processes like appointment scheduling, managing prescriptions and report organization easy which results in reducing the need of additional staff to manage hospital.
- Hosting the website globally by deploying it would eliminate the need for on premise infrastructure. This webpage allows user a fast access as we implemented efficiency, reliability and availability which in turn increases the number of users.

4.1.4.2 Environmental Requirements:

- The Hospital Management System supports a paper free work, no manual records are maintained, in return minimizes the physical files and records.
- The prescriptions are stored in history where user can view them any time without worrying that prescriptions may be lost.
- A software platform reduces the disadvantages of traditional methods and paves a way for modern infrastructures. It encourages modern software tools and technologies to provide eco friendly healthcare facilities.

4.1.4.3 Societal Needs:

- The HMS provides easy access the medical prescriptions, appointment history from any location.
- User does not need to stay in a particular location. They can access the website if they have stable internet connection even in rural areas which helps to increase healthcare facility everywhere.
- It reduces waiting time as we included performance requirement for using the website. This platform enhances the user experience due to its response time and online booking procedure.

4.1.4.4 Political Requirements:

- This web application should support government healthcare policies if they are inspected in the future such as - Ayushman Bharat Digital Mission, etc.
- The website can also be helpful for healthcare databases, vaccination reports nationally if the application is developed further in the future.

- This webpage will be transparent which is easy to handle by everyone in healthcare sector.

4.1.4.5 Ethical Requirements:

- The private information of the users are protected with high priority in the database.
- The user control access ensures only authorized users to modify or delete sensitive information. Admin has the control of CRUD operations and they ensure system rejects any unauthorized logins.
- The prescriptions are only added my trained medical professionals to ensure patients are treated under qualified doctors.

4.1.4.6 Health and Safety Requirements:

- Preventing prescription and appointment errors to avoid any inconvenience to the patient.
- Allows the patient to select the time slot to avoid clashes in the appointment booking. Enables users to view prescription history and appointment history for accurate medical response of the website.
- Displays the appointment in the dashboard so that patient/doctor don't miss the appointment.

4.1.4.7 Sustainability:

- The architecture of the web application is simple and hence it can be maintained and updated any time.

- It supports long term usage to also support future enhancements. This webpage also supports scalability and make sure to expand the specializations are added in the future.
- Any number of components and features can be added with multiple technologies which enables reusability of the application.

4.1.4.8 Legality:

- The application should be able to adhere health data protection laws such as - HIPAA(Health Insurance Portability and Accountability Act) and GDPR(General Data Protection Regulation).
- The webpage should have the user consent policies such as password protection, user profile controls, data access controls.
- Maintaining the login history and audit logs will be useful for legal traceability.

4.1.4.9 Inspectability:

- All the user actions such as appointment booking, prescriptions, login history should be recorded for legal purposes.
- Admin dashboard has all the facilities like adding, deleting and updating the user information which helps to monitor that only authorized users have access for these facilities.
- The user profiles are different and each profile have different functionalities and access controls. Audit logs can be used for performance reviews, system stability and availability.

4.1.5 System Requirements

4.1.5.1 Hardware Requirements

- Windows/MAC laptop
- Central Processing Unit (CPU)
- Graphics Processing Unit (GPU)
- Hard Disk Drives (HDD)
- Random Access Memory (RAM)

4.1.5.2 Software Requirements

- **FrontEnd –** React JS, Bootstrap, CSS
- **BackEnd –** Node JS, Express JS
- **Database –** MongoDB
- **Platform –** Visual Studio Code
- **Local Host browser –** Firefox
- Windows 10 version and above

4.2 SYSTEM DESIGN

4.2.1 System Architecture

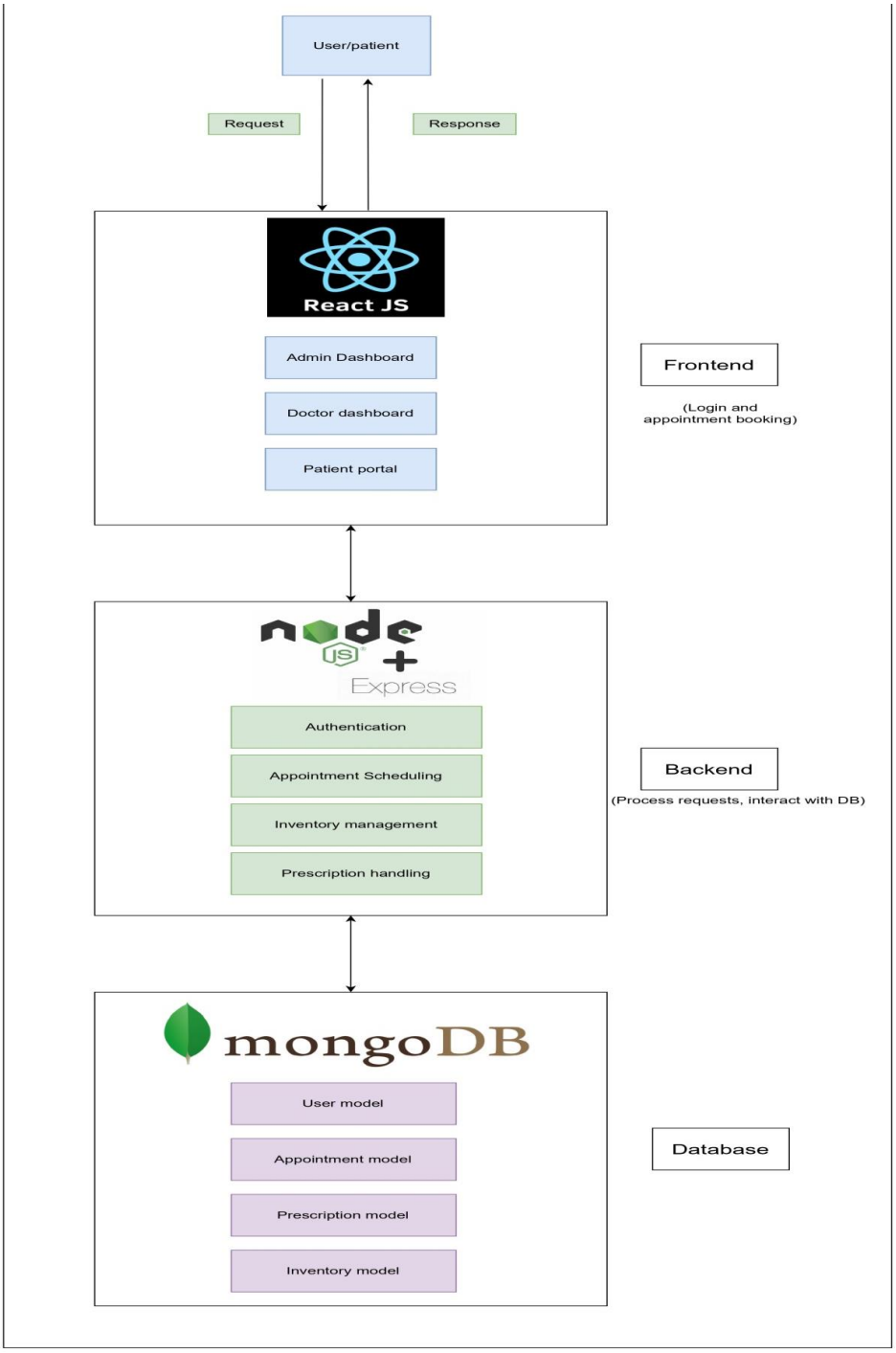

Fig 4.5 System Architecture of HMS

MODULE DESCRIPTION

4.2.1.1 User

User initiates login/registration and creates an account in the website. The request to book an appointment is sent to the frontend and frontend works with backend and database to collect the required data to send appointment availability and booking details to the patient.

4.2.1.2 Frontend

React JS: Serves as the patient and hospital staff's user interface (UI). It shows data from the MongoDB database and communicates with the backend through APIs.

Components:

- **Admin Dashboard:** Provides access to hospital data, medicines, and doctor appointment scheduling management for administrators.
- **Doctor Dashboard:** This allows doctors to manage prescriptions, check appointments, and click their free slots.
- **Patient portal:** Prescriptions, medical history, and appointment scheduling are accessed by the patient. They can also view appointments for today.

4.2.1.3 Backend

Node JS and Express JS: Manages all server-side and makes connection with server side logic, respond to requests made by user.

Components:

- **Authentication:** Uses database to store the credentials of admin, patient and doctor and a secret only the admin has to protect the website.
- **Appointment Scheduling:** Takes care of appointment schedules, time, date and availability of doctor and make sure not to have clashes in between.
- **Prescription handling:** Doctors are able to write prescriptions in their logins.

4.2.1.4 Database

Mongo DB: Tracks all of each application's data, including patient data, appointment and physician schedules, prescription data, and inventory data.

Components:

- **User Model:** Gives information (name, contact details, role, etc.) regarding Admins, Doctors, and Patients.
- **Appointment Model:** Maintains patient, doctor, date, and status data for every appointment.
- **Prescription Model:** Stores any prescriptions given by doctor in their respective logins to check in future.
- **Inventory Model:** Keeps tabs on medicines, updating the medicines, adding the medicines, providing the price of the medicine.

4.2.2 Detailed Design

4.2.2.1 Use Case Diagram using Star UML application

Actors – Patient, Doctor, Admin, Database

Usecases for each actor:

Admin – Add user, delete user, Update details, Logout, Login, Signup, Manage schedule

Patient – Login, Signup, Logout, Book appointment(Select time, date, doctor)

Doctor – Select their free slots, Login, Signup, Logout, Check patient, add prescriptions

Database – Store user credentials, Gather patient information

Use case diagram is designed using star uml application. First identify actors and their use cases. Open the application and select the model – use case diagram and continue to draw the diagram which will result as below:

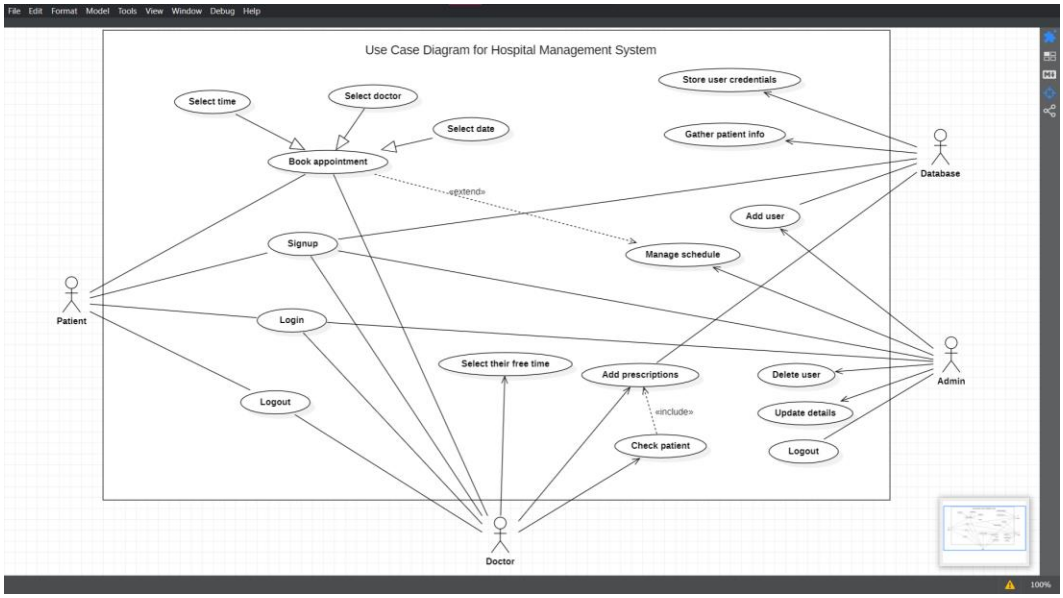

Fig 4.6 Use case diagram of HMS

4.2.2.2 Class Diagram using Star UML application

The boxes represent classes.

Classes – Patient, Doctor, Admin, Prescription, Department, Book appointment, Database

Classes have attributes and operations. The first half of the box represents attributes. Attributes are information of the class. Operations are represented below the attributes. Operations describe the activity of the class and what kind of role they play in the application.

After identifying these, classes are connected using connections like association, dependency, directed association.

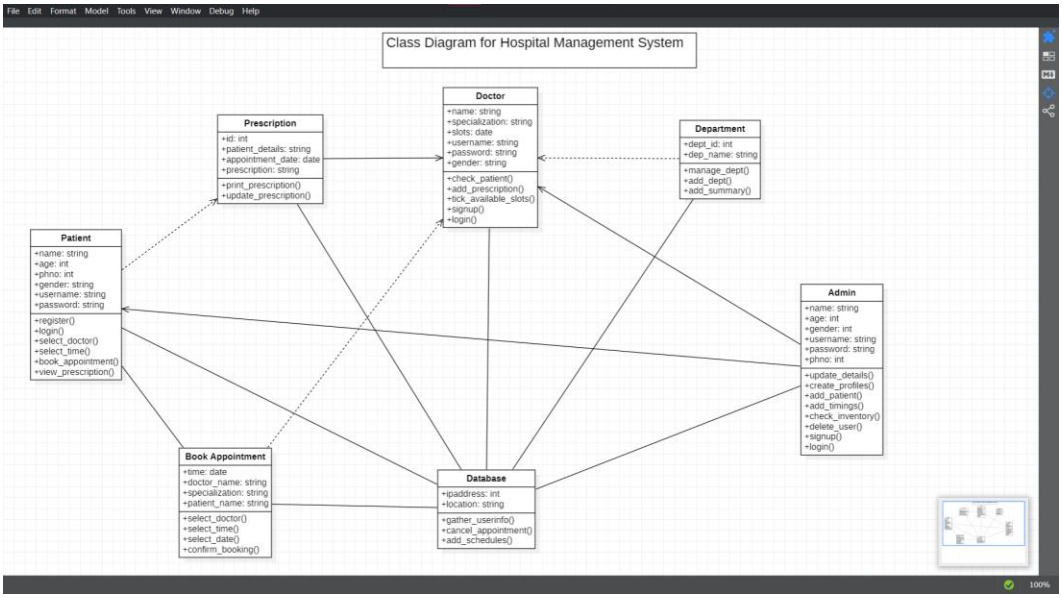

Fig 4.7 Class diagram of HMS

4.2.2.3 State Chart Diagram using Star UML application

State chart diagram represents the behavior of the whole application aspect. It is also known as behavioural view of the application.

State chart diagram is started with Initial state and simple states/actions of the website are connected through transition (event). After all the actions of each user is completed they logout. The logout activity is connected to Final state at the end to confirm that state chart diagram is completed.

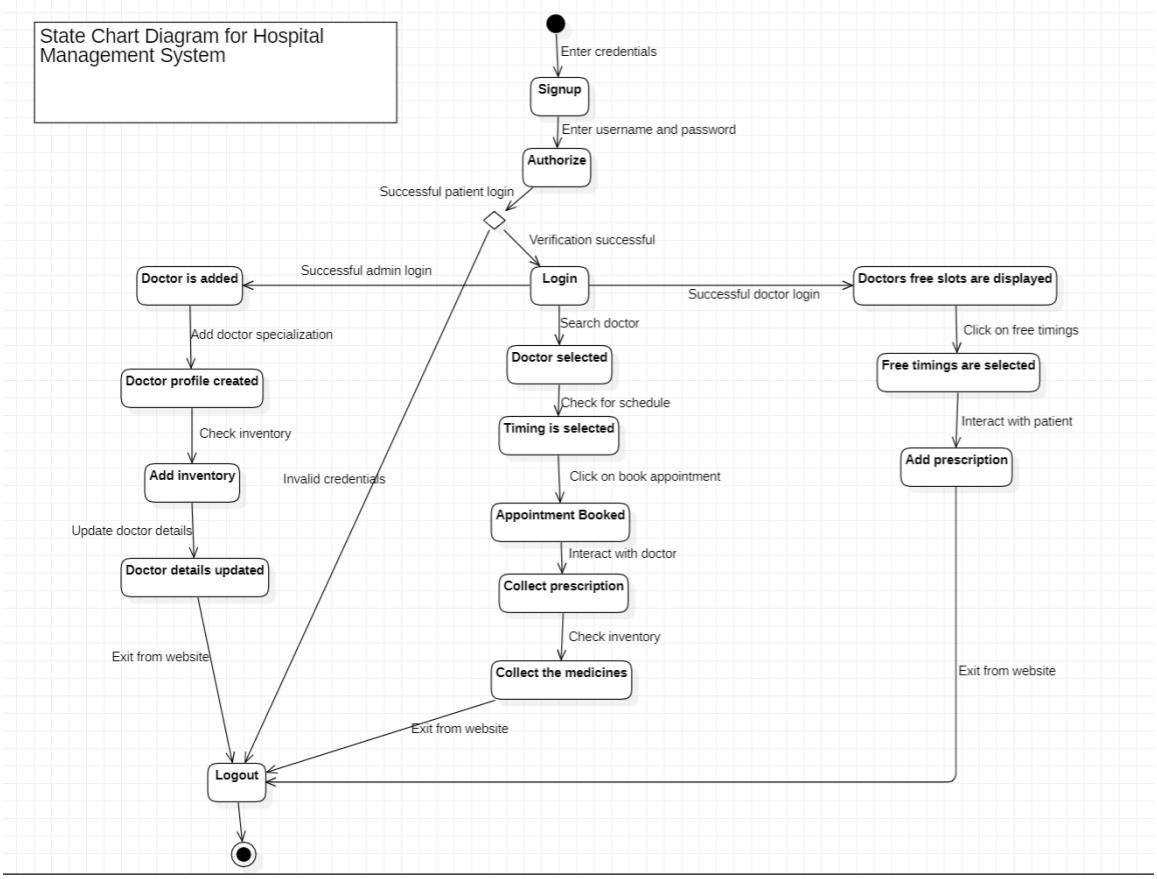

Fig 4.8 State chart diagram of HMS

4.2.2.4 Activity Diagram using Star UML application

This diagram represents behavior of individual use case. It is specifically useful for developers to create the modules. All the step by step activities are mentioned here.

All the actors are represented by swim lane and attached together to start the activity diagram.

Activity diagram is started by Initial activity, actions are connected to each other according to the functionalities under actors swim lane till the final activity.

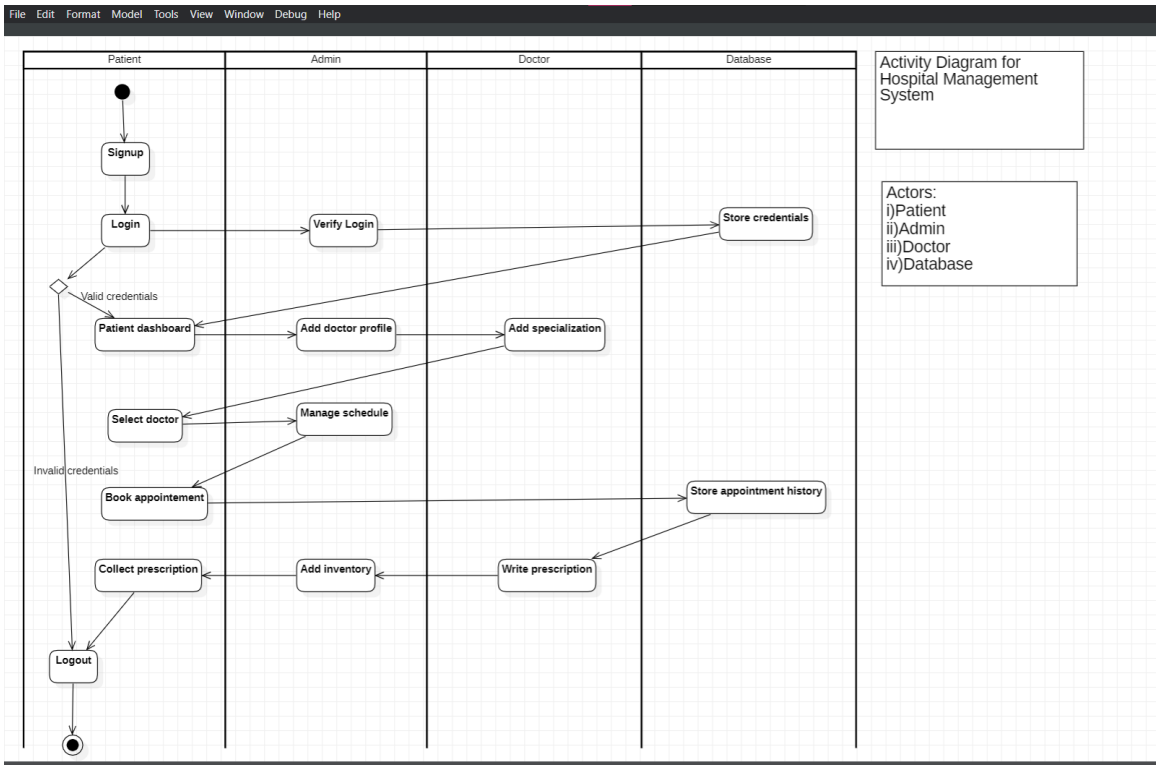

Fig 4.9 Activity diagram of HMS

4.2.2.5 Sequence Diagram using Star UML application

Sequence diagram is a interaction diagram between the lifelines (main actors of the application). It shows how the communications occur between object and entity.

The object/entity is called a lifeline. Request message is sent to the life line and the other objects respond according to the request message. Lifeline is known for how long the communication is going on.

There is also self message symbol for the object to communicate with itself.

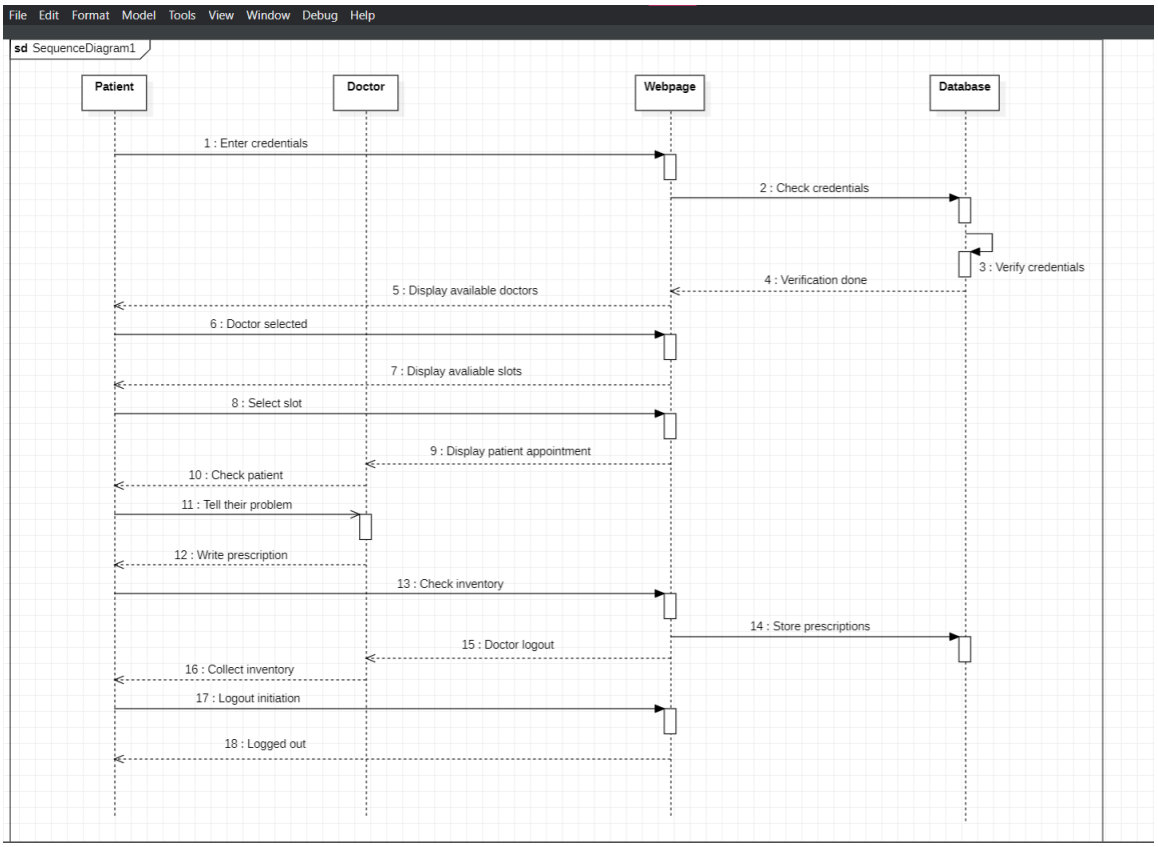

Fig 4.10 Sequence diagram of HMS

4.2.2.6 Communication Diagram using Star UML application

Communication diagram is the opposite of sequence diagram. It is also known as Collaboration diagram.

Lifelines are separated from each other and wait for request messages to respond accordingly. Collaboration diagram also has self message. All the features of sequence diagram can be used for this diagram.

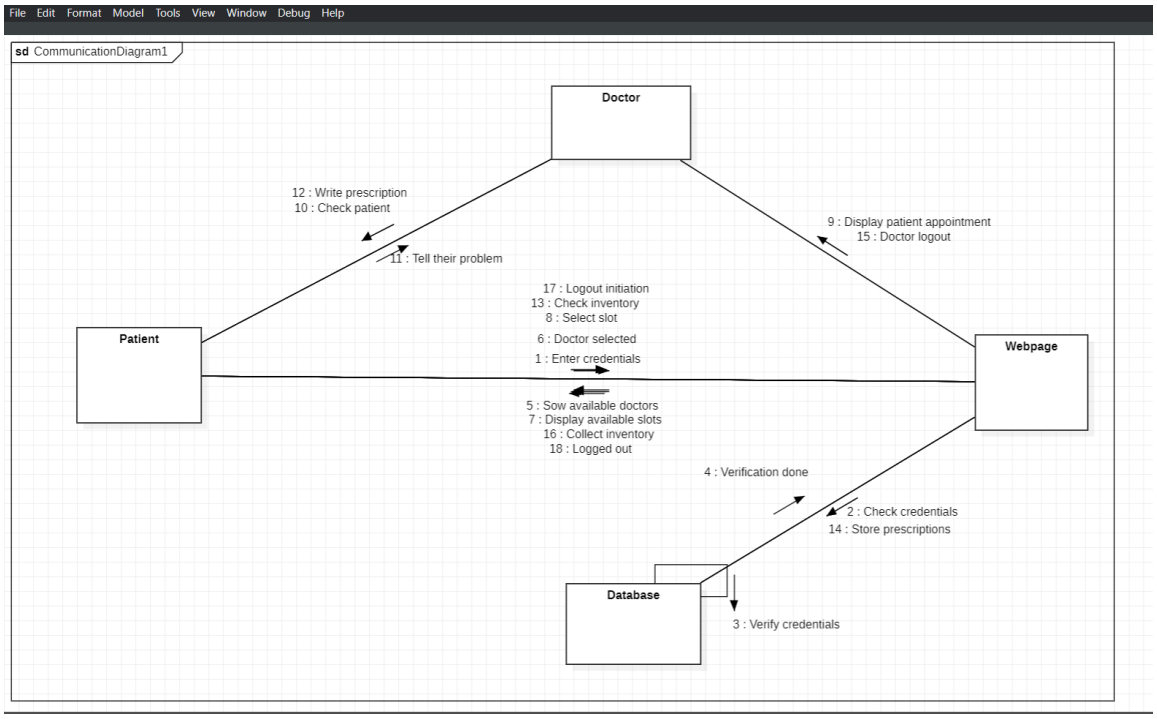

Fig 4.11 Collaboration/Communication diagram of HMS

5. IMPLEMENTATION AND TESTING

To implement the Hospital Management System, the following tools and technologies have been used:

5.1 Front End - React JS, CSS and Bootstrap

This project has a user friendly interface for different profiles. There are 3 different logins – Admin, Doctor and Patient. Every component is segregated as Admin, Doctor and Patient to ensure all the functionalities are integrated properly to each profile. React JS project is classified into client and server. React is used for client. To install react project, run the command “npx create-react-app .” in the VS code. The node modules are installed and we are ready to perform frontend logic. I developed user-friendly dashboards, appointment booking forms, prescription management interfaces, and user account management pages. The other packages required for the implementation can be installed accordingly by “npm install” command.

5.2 Back End – Node JS, Express JS

Node JS is installed in the VS Code by using npm install or directly installing latest version from the Google. Express JS is a Node JS framework used to build the application offering middleware, routing, etc. The backend is divided into 4 parts mainly - Controllers, models, middlewares, and routes. Controllers have the main backend logic of the application. It manages request responses. Middlewares controls the authentication purposes, like to authenticate the users of application or it authenticates the requests coming. Models has their own backend logics like doctors profile, admin profile etc. Routes establishes the communication between backend and frontend by having the endpoints if the frontend pages.

5.3 Database – MongoDB, Mongoose

Create a MongoDB account using and select the dropdown menu to create a new project. It ensures to store the user credentials and their information. Mongoose helps to simplify the interaction with the database. Installation is done by “npm install mongoose” command

5.4 Dotenv

It is a module that enables the use of environment variables in a Node.js application. Dotenv is used to manage all our sensitive configuration variables, such as database connection strings, API keys, port number and secret tokens.

5.5 Axios

Axios is a node package which is used to fetch the doctor prescription, appointments, and their respective information from the database. And axios is also used in user authentication.

5.6 Output Screenshots

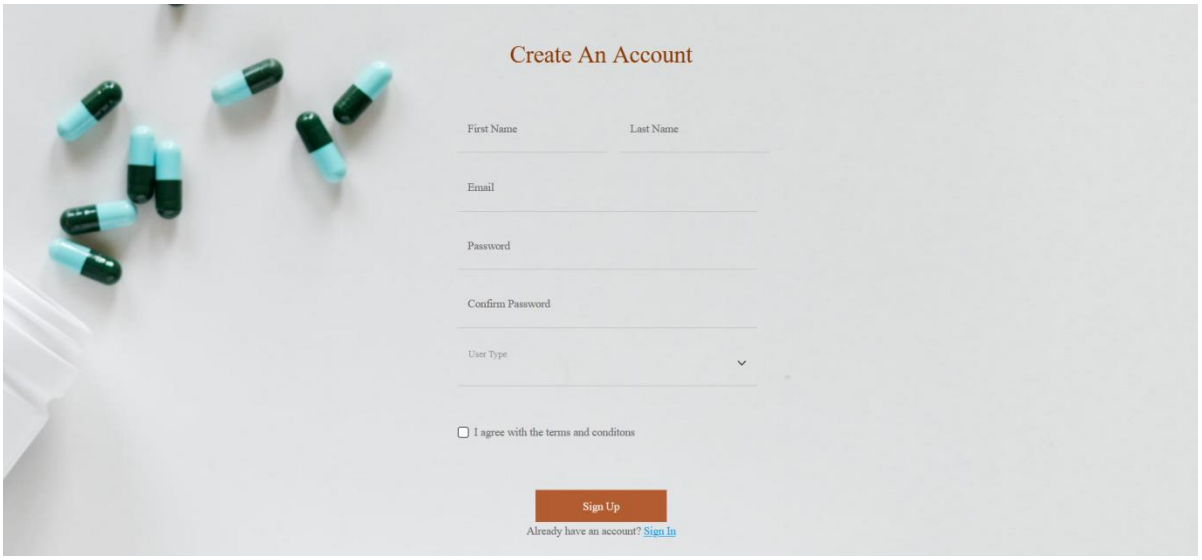

Fig 5.1 Signup Page

Login page

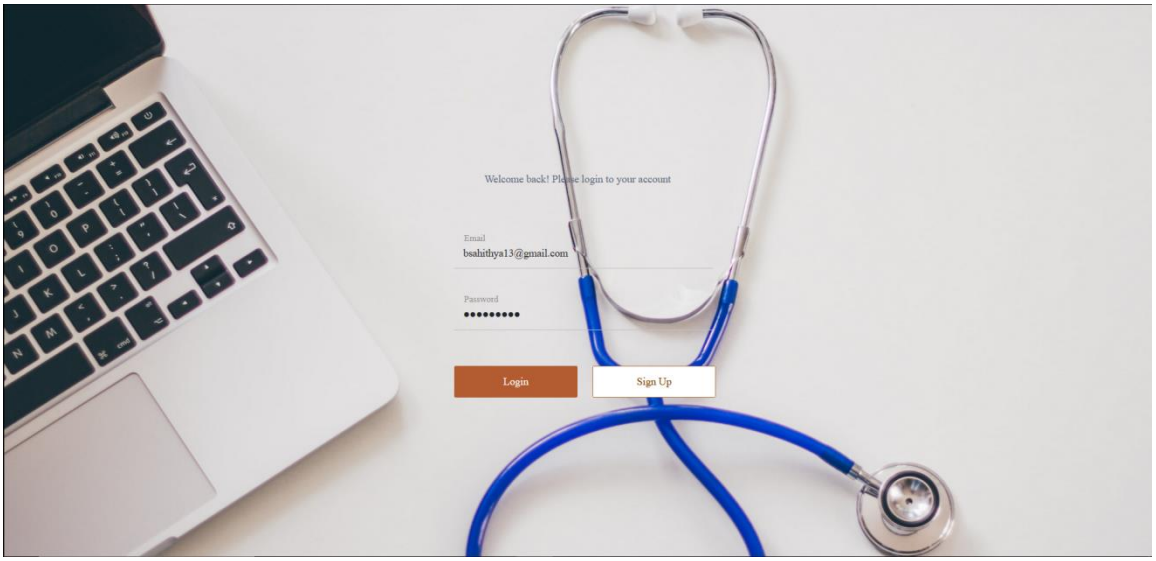

Fig 5.2 Login page

Admin Dashboard and its functionalities

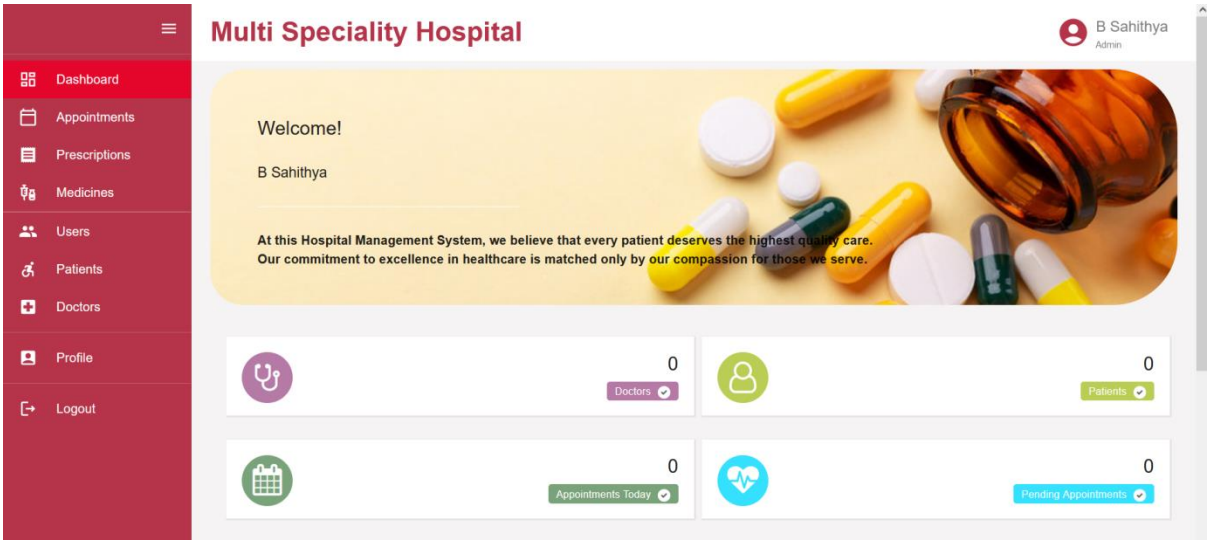

Fig 5.3 Admin Dashboard

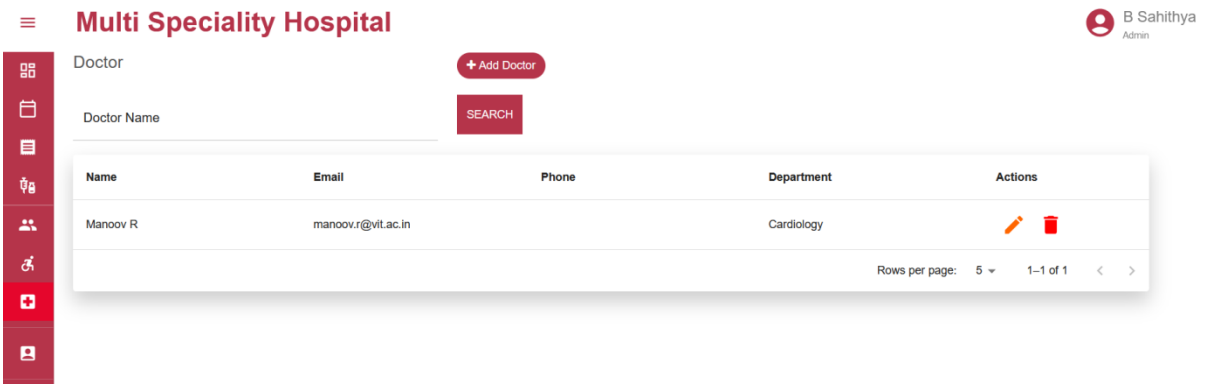

Fig 5.4 Add doctor page in admin's profile

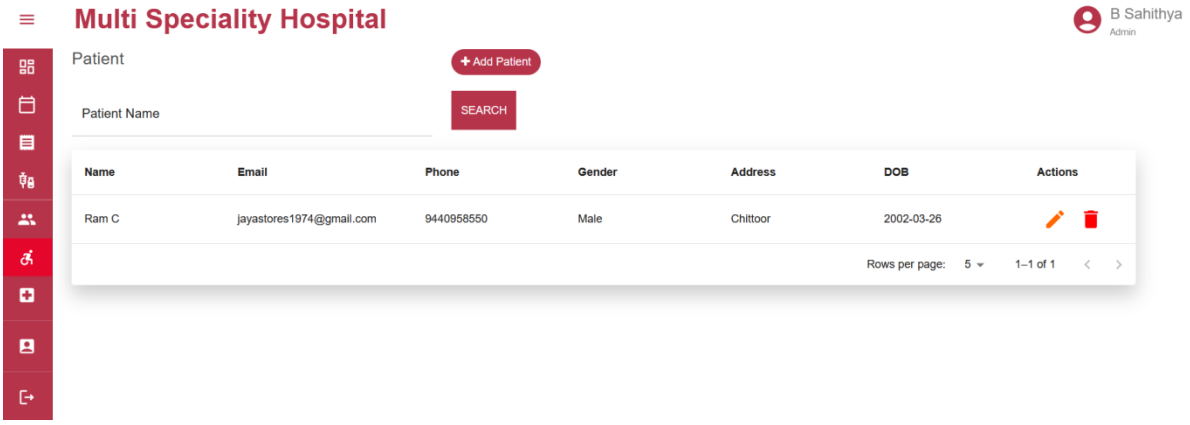

Fig 5.5 Add patient module in admin's login

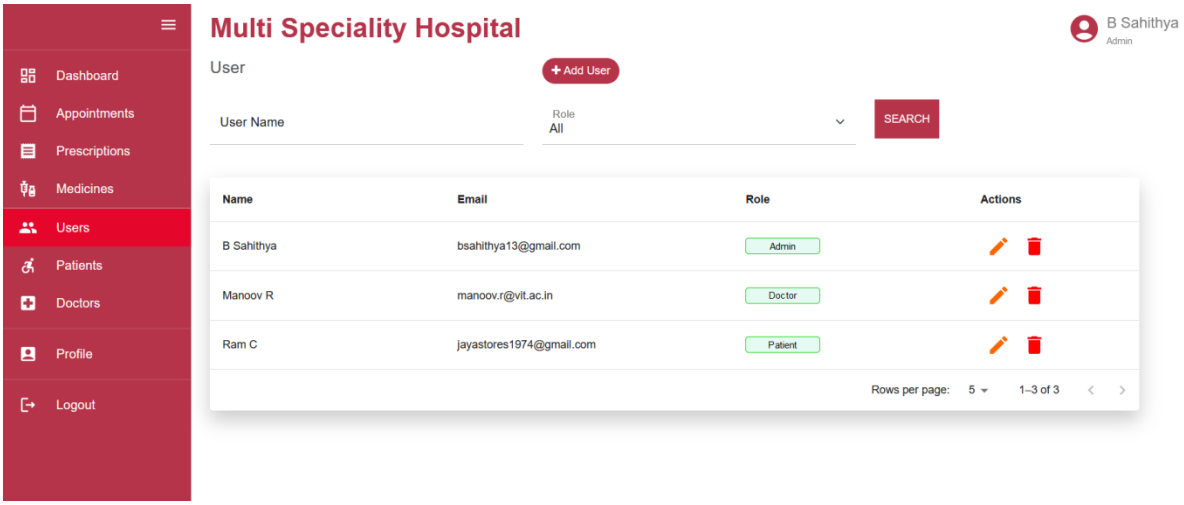

Fig 5.6 List of users

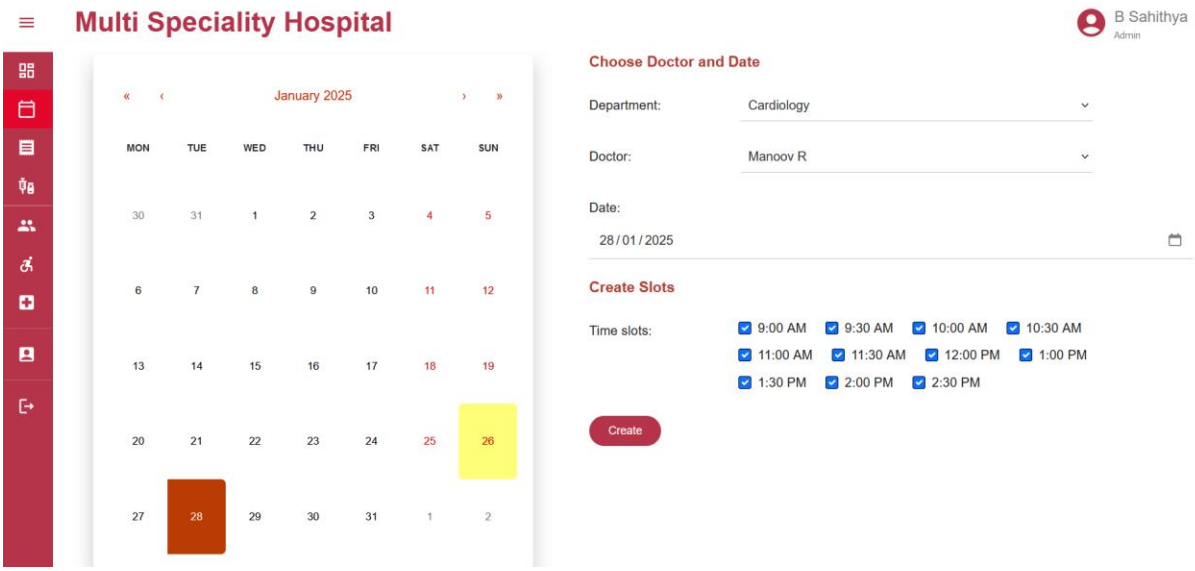

Fig 5.7 Add doctor, department, date and create slots

Delete user

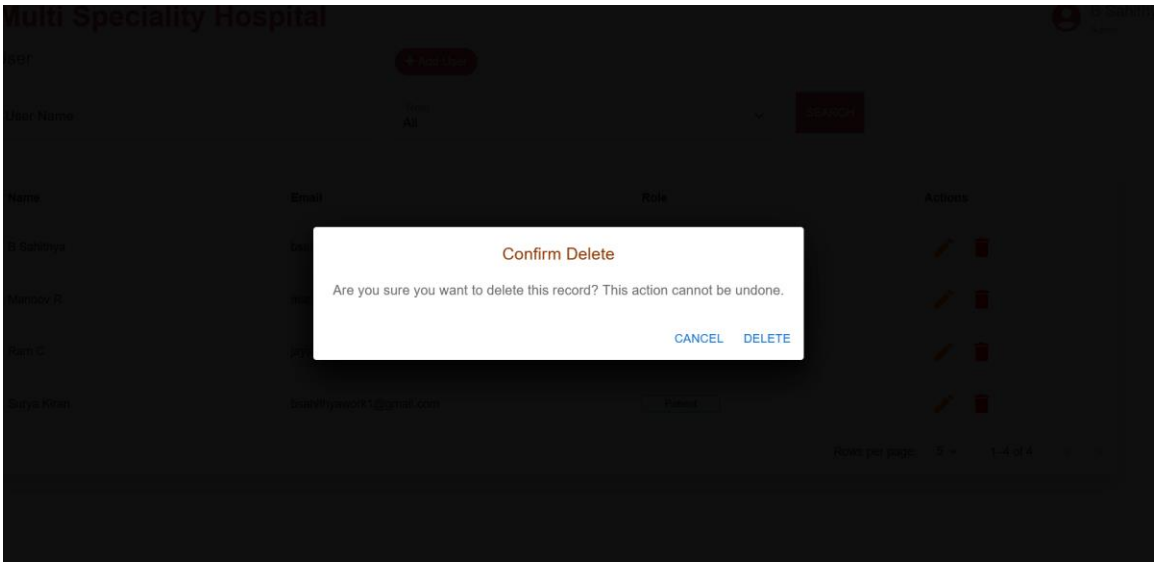

Fig 5.8 Delete user option

Doctor’s dashboard

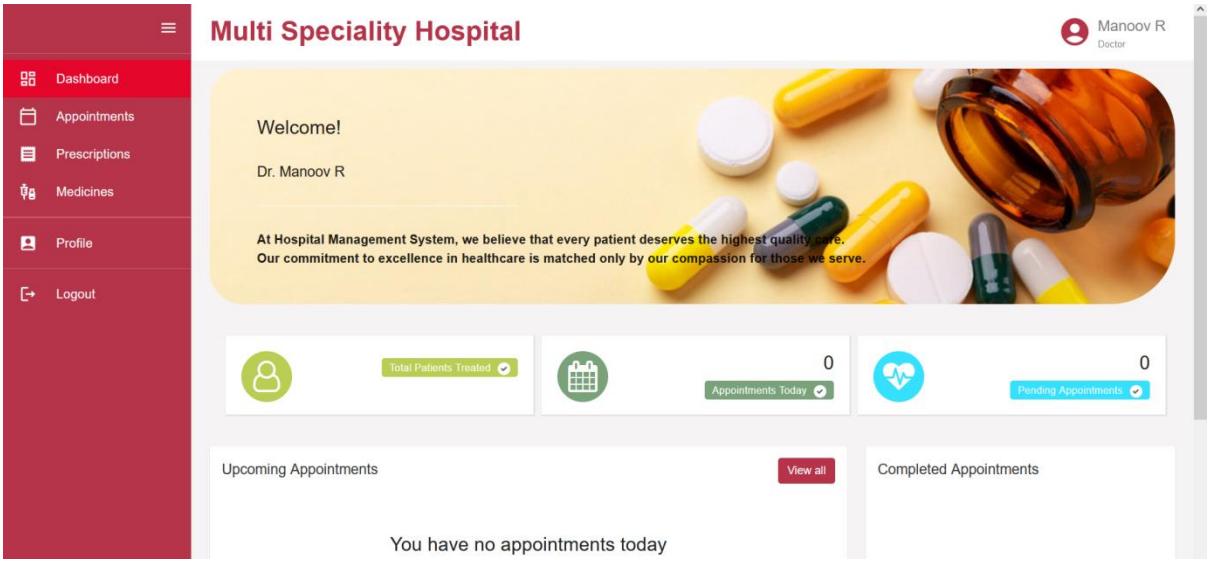

Fig 5.9 Doctor’s login

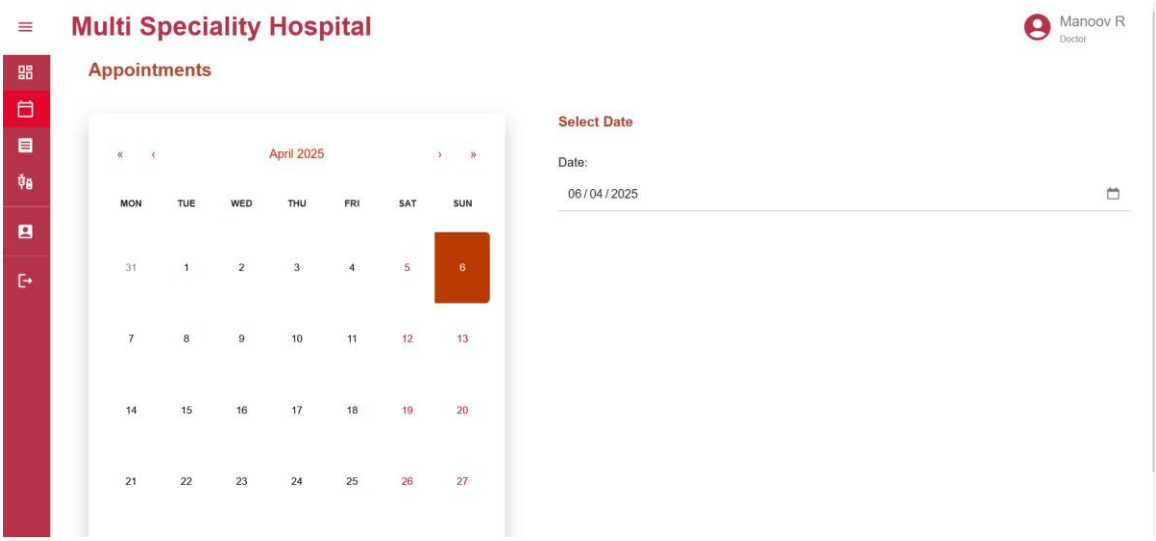

Fig 5.10 Doctors select their available date

Doctor’s Profile

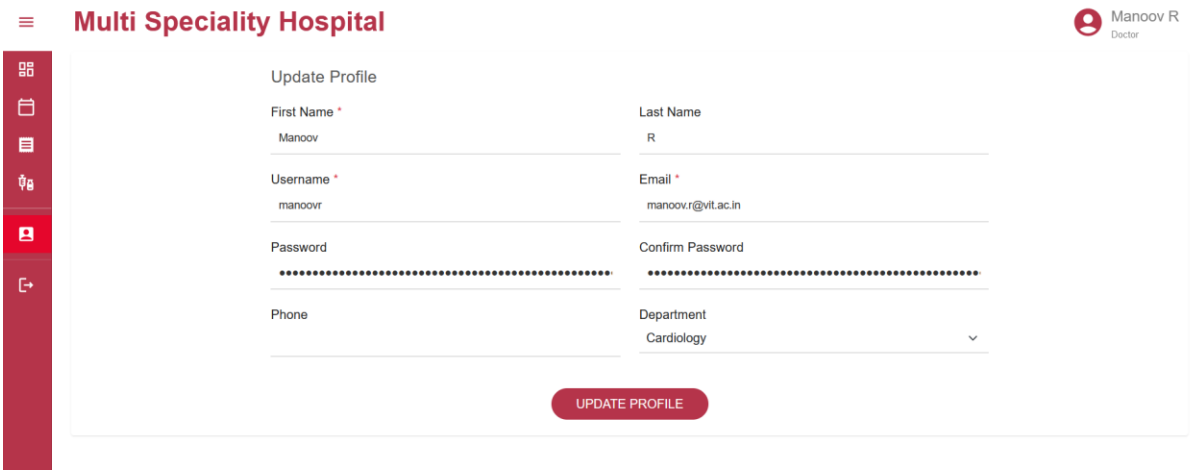

Fig 5.11 Doctor’s profile page

Patient’s Dashboard

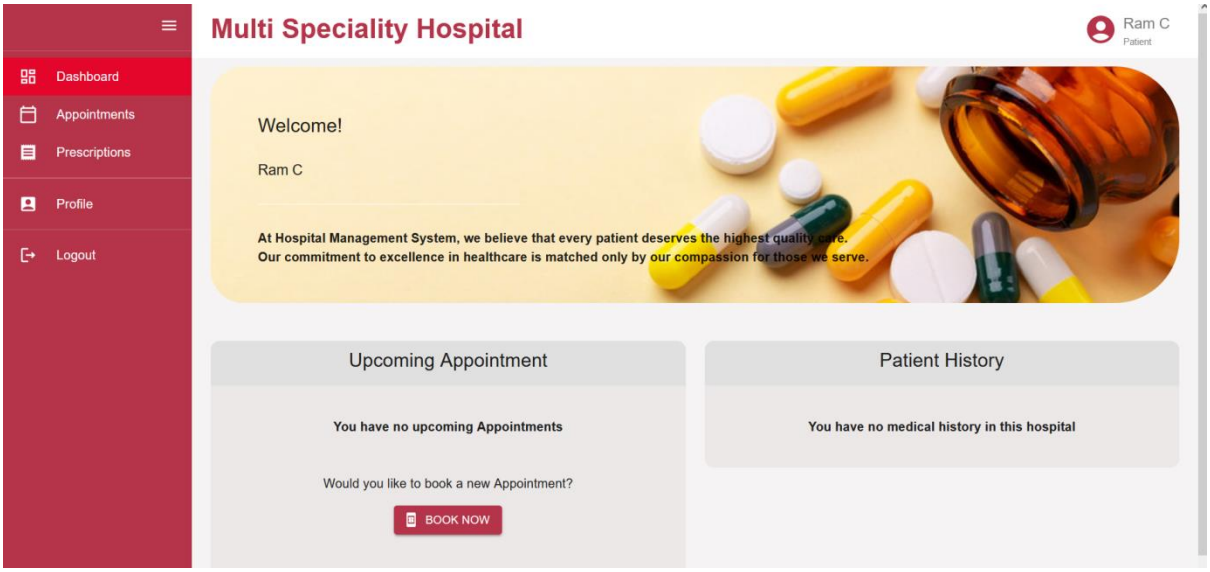

Fig 5.12 Patient login

Patient’s profile

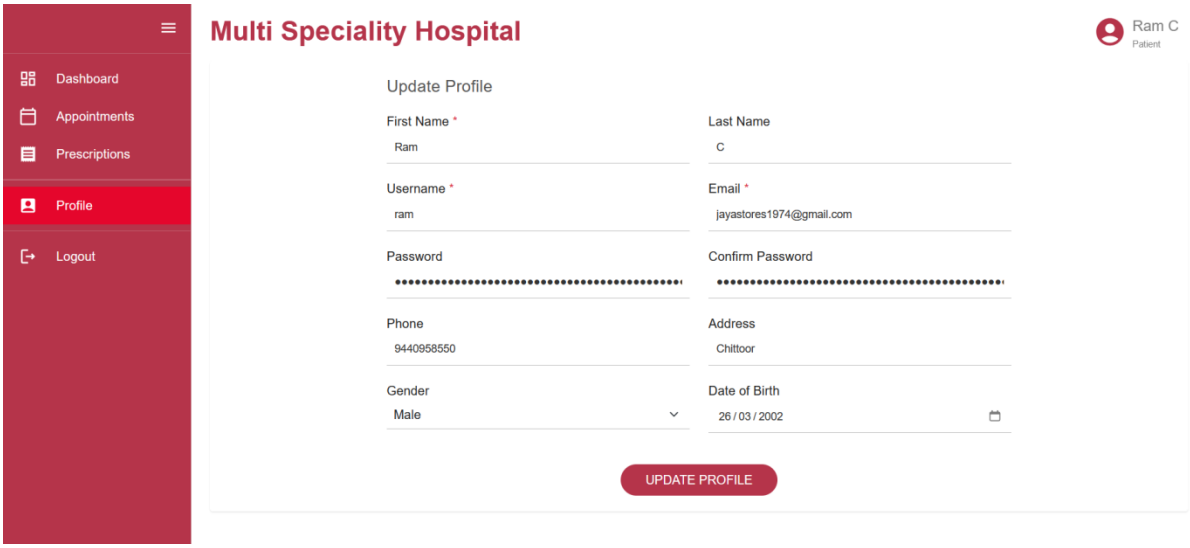

Fig 5.13 Patient profile page

Add medicine – Admin

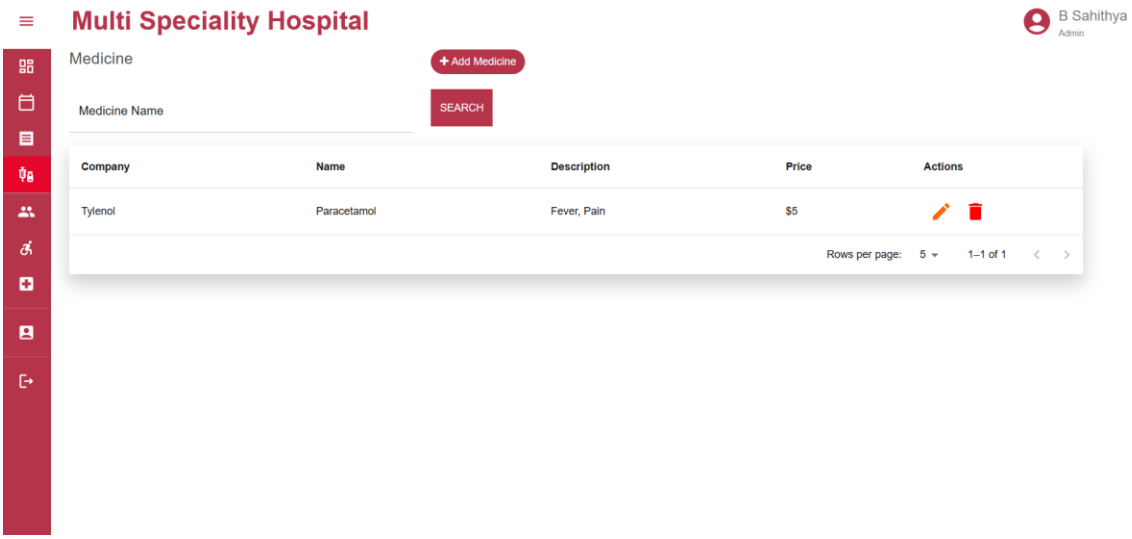

Fig 5.14 Adding inventory

Update medicine – Admin

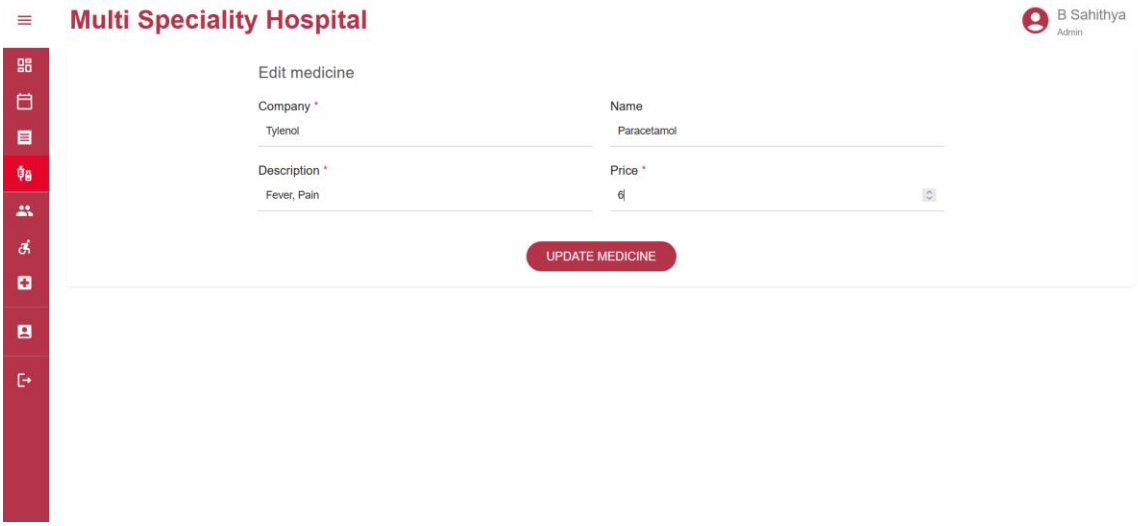

Fig 5.15 Updating inventory

Update patient details – Admin

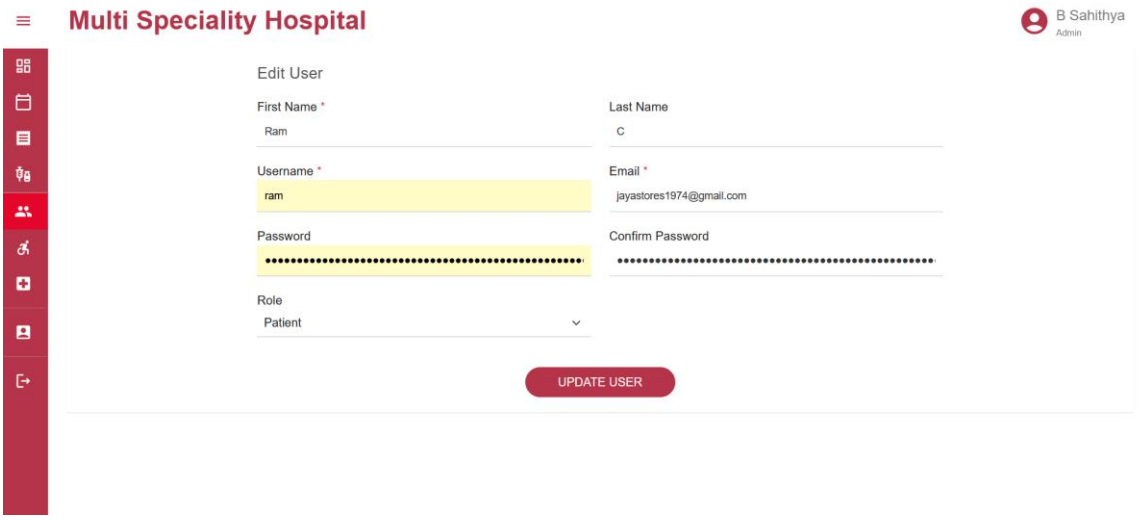

Fig 5.16 Update user details

6. RESULTS AND DISCUSSION

In this project, I have created an appointment booking system which helps patients to login to the website and book the doctor appointment and get the prescription given by the doctor. The results of this project include:

i. All the users – doctors, admins, and patients have a different dashboard based on the permissions and each has different benefits.

ii. A sign up page is created for new user. After a successful registration, user is ready to login to the website.

iii. After successful authentication, user can view the available doctors and their schedule. Users can select the doctor of their choice and book the appointment.

iv. Patient's dashboard has the facility to check the schedule and available slots of the doctor to schedule an appointment.

v. Admins can add, update and delete the user details which provide the authority of performing CRUD operation to the website.

vi. User profile is categorized as 3 profiles – Admin, Doctor and patient. Admin is the one who operate the prescriptions, inventory, and manage user details. Doctor can select their free time to check the patient and add the prescription. Patient can book the appointment and view the prescription.

7. CONCLUSION AND FUTURE WORK

The Hospital Management System implemented with the MERN stack effectively solves major issues such as appointment scheduling, patient record management, prescription management, and medicine inventory management. Automating repetitive tasks and providing role-based access, the system improves the efficiency, accuracy, and accessibility of healthcare services. It offers an easy-to-use interface for admins, doctors, and patients, making the experience smooth across all modules. The system helps in improved organization of data, less administrative burden, and enhanced patient care through prompt access to medical information.

Future Work:

- **Billing Gateway:** For development of an online payment module for appointment scheduling and inventory.
- **Laboratory Page:** In order to install a module for the patient to schedule for lab tests and produce reports and consult doctor. In order to schedule for lab testing.
- **Android and iOS app:** Developing an app for android and iOS to make the application more accessible so that individuals can use it conveniently in a vast array of various people.
- **Multilingual Support:** To include regional languages such that users who cannot read English also be able to use the application.

Annexure – I - Sample Code

7.1. FrontEnd

AdminProfile.js

```
import React, { useEffect, useState, useContext } from 'react';
import { NavLink } from 'react-router-dom'
import { useNavigate, useParams } from "react-router-dom";
import ErrorDialogueBox from '../MUIDialogueBox/ErrorDialogueBox';
import axios from "axios";
import Box from '@mui/material/Box';
import { UserContext } from '../../Context/UserContext'
function AdminProfile() {
  const navigate = useNavigate();
  const { currentUser } = useContext(UserContext);
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [email, setEmail] = useState('');
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [userId, setUserId] = useState('');
  const [passwordMatchDisplay, setPasswordMatchDisplay] = useState('none');
  const [patientId, setPatientId] = useState('');
  const [passwordValidationMessage, setPasswordValidationMessage] = useState('');
  const [errorDialogueBoxOpen, setErrorDialogueBoxOpen] = useState(false);
  const [errorList, setErrorList] = useState([]);
  const handleDialogueOpen = () => {
    setErrorDialogueBoxOpen(true)
  };
  const handleDialogueClose = () => {
    setErrorList([]);
    setErrorDialogueBoxOpen(false)
  };
  useEffect(() => {
    getAdminById();
  }, []);
  const getAdminById = async () => {
    let adminUserId = currentUser.userId;
    const response = await axios.get(`http://localhost:3001/profile/admin/${adminUserId}`);
    setUserId(response.data._id);
    setFirstName(response.data.firstName);
    setLastName(response.data.lastName);
    setEmail(response.data.email);
    setUsername(response.data.username);
    setPassword(response.data.password);
    setConfirmPassword(response.data.password);
  };
  const updateAdmin = async (e) => {
    e.preventDefault();
    try {
      await axios.patch(`http://localhost:3001/profile/admin/${userId}`, {
        firstName,
        lastName,
        username,
        email,
        password,
        confirmPassword
      });
      navigate("/profile");
    } catch (error) {
      console.log(error.response.data.errors);
      //Display error message
      setErrorList(error.response.data.errors);
```

```
      handleDialogueOpen();
    }
  };
  useEffect(() => {
    if ((typeof password !== 'undefined') && password.length > 0 && password?.trim()?.length <= 6) {
      setPasswordValidationMessage('Password Length must be greater than 6 characters');
    }
    else {
      setPasswordValidationMessage('');
    }
    if (password === confirmPassword) {
      setPasswordMatchDisplay('none');
    }
    else {
      setPasswordMatchDisplay('block');
    }
  }, [password, confirmPassword])
  return (
    <Box component="main" sx={{ flexGrow: 1, p: 3 }}>
      <div className="page-wrapper">
        <div className="content">
          <div className="card-box">
            <div className="row">
              <div className="col-lg-8 offset-lg-2">
                <h3 className="page-title" style={{backgroundColor: '#b5344a', color: 'white', border: 'none'}}>Update Profile</h3>
              </div>
            </div>
            <div className="row">
              <div className="col-lg-8 offset-lg-2">
                <form id="addAdminForm" name='addAdminForm' onSubmit={updateAdmin}>
                  <div className="row">
                    <div className="col-sm-6">
                      <div className="form-group">
                        <label>First Name <span className="text-danger">*</span></label>
                        <input name="firstName" className="form-control" type="text" required value={firstName} onChange={(event) =>
setFirstName(event.target.value)} />
                      </div>
                    </div>
                    <div className="col-sm-6">
                      <div className="form-group">
                        <label>Last Name</label>
                        <input name="lastName" className="form-control" type="text" required value={lastName} onChange={(event) =>
setLastName(event.target.value)} />
                      </div>
                    </div>
                    <div className="col-sm-6">
                      <div className="form-group">
                        <label>Username <span className="text-danger">*</span></label>
                        <input name="username" className="form-control" type="text" required value={username} onChange={(event) =>
setUsername(event.target.value)} />
                      </div>
                    </div>
                    <div className="col-sm-6">
                      <div className="form-group">
                        <label>Email <span className="text-danger">*</span></label>
                        <input name="email" className="form-control" type="email" required value={email} onChange={(event) =>
setEmail(event.target.value)} />
                      </div>
                    </div>
                    <div className="col-sm-6">
                      <div className="form-group">
                        <label>Password</label>
                        <input name="password" className="form-control" type="password" value={password} onChange={(event) =>
setPassword(event.target.value)} />
                      </div>
                    </div>
                    <div className="col-sm-6">
                      <div className="form-group">
                        <label>Confirm Password</label>
```

```
                    <input name="confirmPassword" className="form-control" type="password" value={confirmPassword}
onChange={(event) => setConfirmPassword(event.target.value)} />
                  </div>
                </div>
              </div>
              <div className="m-t-20 text-center">
                <button type="submit" className="btn btn-primary submit-btn" style={{backgroundColor: '#b5344a', color: 'white',
border: 'none'}}>Update </button>
              </div>
            </form>
          </div>
        </div>
      </div>
    </div>
    <ErrorDialogueBox
      open={errorDialogueBoxOpen}
      handleToClose={handleDialogueClose}
      ErrorTitle="Error: Edit Admin"
      ErrorList={errorList}
    />
  </div>
 </Box>
 )
}
export default AdminProfile;
```

AdminDashboard.js

```
import styles from './Dashboard.module.css';
import { React, useState, useEffect, useContext } from 'react';
import { styled, useTheme } from '@mui/material/styles';
import Box from '@mui/material/Box';
import axios from "axios";
import { NavLink } from 'react-router-dom';
import moment from "moment"
import { UserContext } from '../../Context/UserContext'
export default function AdminDashboard() {
  const [doctorCount, setDoctorCount] = useState(0);
  const [patientCount, setPatientCount] = useState(0);
  const [appsTodayCount, setAppsTodayCount] = useState(0);
  const [pendingAppsTodayCount, setPendingAppsTodayCount] = useState(0);
  const [bookedAppointments, setBookedAppointments] = useState([]);
  const [doctors, setdoctor] = useState([]);
  const { currentUser } = useContext(UserContext);
  const getUserCountByRole = async (userType) => {
    const response = await axios.post(`http://localhost:3001/count/users`,
      {
        'userType': userType
      },
      {
        headers: {
          authorization: `Bearer ${localStorage.getItem("token")}`
        }
      }
    );
    let count = response.data.count
    if (count) {
      if (userType == "Doctor")
        setDoctorCount(count);
      else if (userType == "Patient")
        setPatientCount(count);
    }

  };
  const getAppointmentCount = async () => {
    const response = await axios.get(`http://localhost:3001/count/appointments`,
```

```
        {
            headers: {
                authorization: `Bearer ${localStorage.getItem("token")}`
            }
        }
    );
    if (response?.data?.totalAppointments) {
        setAppsTodayCount(response?.data?.totalAppointments);
    }
    if (response?.data?.pendingAppointments) {
        setPendingAppsTodayCount(response?.data?.pendingAppointments)
    }
}

const getBookedSlots = async () => {
    // console.log(moment(new Date()).format('YYYY-MM-DD'))
    let response = await axios.post(`http://localhost:3001/appointments`,
        {
            'isTimeSlotAvailable': false,
            'appDate': moment(new Date()).format('YYYY-MM-DD')
        },
        {
            headers: {
                authorization: `Bearer ${localStorage.getItem("token")}`
            }
        }
    );
    if (response.data.message == "success") {
        let aptms = response.data.appointments;
        console.log("aptms", aptms);
        setBookedAppointments(aptms);
    }
}
const getdoctors = async () => {
    const response = await axios.get("http://localhost:3001/doctors");
    setdoctor(response.data);
};
useEffect(() => {
    getUserCountByRole("Doctor");
    getUserCountByRole("Patient");
    getAppointmentCount()
    getBookedSlots();
    getdoctors();
}, []);

return (
    <Box className={styles.dashboardBody} component="main" sx={{ flexGrow: 1, p: 3 }}>
        <div id={styles.welcomeBanner}>
            <div className='text-black'>
            <b> <h3 >Welcome!</h3>
                <br/>
                <h4>{currentUser.firstName} {currentUser.lastName}</h4>
                <br/>
                <div class={styles.horizontalLine}></div>
                At this Hospital Management System, we believe that every patient deserves the highest quality care.
                <br/>
                Our commitment to excellence in healthcare is matched only by our compassion for those we serve.</b>

            </div>

        </div>
        <div className={styles.statCardGrid}>
            <div className={["", styles.statCard].join(" ")}>
                <div className={styles.dashWidget}>
                    <span className={styles.dashWidgetBg1}><i className="fa fa-stethoscope" aria-hidden="true"></i></span>
                    <div className={[" ", styles.dashWidgetInfo].join(" ")} >
                        <h3 className={styles.dashWidgetInfoH3}>{doctorCount}</h3>
                        <span className={styles.widgetTitle1}>Doctors <i class="fa fa-check" aria-hidden="true"></i></span>
                    </div>
                </div>
```

```
            </div>
            <div className={["", styles.statCard].join(" ")}>
               <div className={styles.dashWidget}>
                  <span className={styles.dashWidgetBg2}><i className="fa fa-user-o" aria-hidden="true"></i></span>
                  <div className={[" ", styles.dashWidgetInfo].join(" ")} >
                     <h3 className={styles.dashWidgetInfoH3}>{patientCount}</h3>
                     <span className={styles.widgetTitle2}>Patients <i class="fa fa-check" aria-hidden="true"></i></span>
                  </div>
               </div>
            </div>
            <div className={["", styles.statCard].join(" ")}>
               <div className={styles.dashWidget}>
                  <span className={styles.dashWidgetBg3}><i className=" fa fa-calendar" aria-hidden="true"></i></span>
                  <div className={[" ", styles.dashWidgetInfo].join(" ")} >
                     <h3 className={styles.dashWidgetInfoH3}>{appsTodayCount}</h3>
                     <span className={styles.widgetTitle3}>Appointments Today <i class="fa fa-check" aria-
hidden="true"></i></span>
                  </div>
               </div>
            </div>
            <div className={["", styles.statCard].join(" ")}>
               <div className={styles.dashWidget}>
                  <span className={styles.dashWidgetBg4}><i className="fa fa-heartbeat" aria-hidden="true"></i></span>
                  <div className={[" ", styles.dashWidgetInfo].join(" ")} >
                     <h3 className={styles.dashWidgetInfoH3}>{pendingAppsTodayCount}</h3>
                     <span className={styles.widgetTitle4}>Pending Appointments <i class="fa fa-check" aria-
hidden="true"></i></span>
                  </div>
               </div>
            </div>
         </div>

         <div className="row ">
            <div className="col-12 col-lg-8 col-xl-8">
               <div className="card appointment-panel">
               <div className="card-header">
   <h4 className="card-title d-inline-block">Upcoming Appointments</h4>
   <NavLink to="/appointments" className="btn btn-primary float-end" style={{backgroundColor: '#b5344a'}}>View
all</NavLink>
</div>

                  <div className="card-body">
                     <div className="table-responsive">
                        <table className="table mb-0">
                           <thead className="d-none">
                              <tr>
                                 <th>Patient Name</th>
                                 <th>Doctor Name</th>
                                 <th>Timing</th>
                                 <th className="text-right">Status</th>
                              </tr>
                           </thead>
                           <tbody>
                              {bookedAppointments.map((apt) => {
                                 return (
                                    <tr>
                                       <td className={styles.appointmentTableTd}>
                                          <a className="avatar" href="">{apt?.patientId?.userId?.firstName?.charAt(0)}</a>
                                          <h2 className='ps-3'><a href="">{apt?.patientId?.userId?.firstName}
{apt?.patientId?.userId?.lastName} <span>{apt?.patientId?.address}</span></a></h2>
                                       </td>
                                       <td>
                                          <h5 className="time-title p-0">Appointment With</h5>
                                          <p>Dr. {apt?.doctorId?.userId?.firstName} {apt?.doctorId?.userId?.lastName}</p>
                                       </td>
                                       <td>
                                          <h5 class="time-title p-0">Timing</h5>
                                          <p>{apt?.appointmentTime}</p>
                                       </td>
                                       {}
```

```
                                    </tr>
                                )
                            })
                            }


                        </tbody>
                    </table>
                    {(!bookedAppointments || bookedAppointments?.length === 0) &&
                        <h3 className='mt-5 text-center '>
                            You have no appointments today
                        </h3>
                    }
                </div>
            </div>
        </div>
        <div class="col-12 col-lg-4 col-xl-4">
            <div class="card member-panel">
                <div class="card-header bg-white">
                    <h4 class="card-title mb-0">Doctors</h4>
                </div>
                <div class="card-body">
                    <ul class="contact-list">
                        {doctors && doctors.map((doc) => {
                            return (
                                <li>
                                    <div class="contact-cont">
                                        <div class="float-left user-img m-r-10">
                                            {/*                                         </div>
                                        <div class="contact-info">
                                            <span class="contact-name text-ellipsis">{doc.userId?.firstName} {doc.userId?.lastName}</span>
                                            <span class="contact-date">{doc.department} </span>
                                        </div>
                                    </div>
                                </li>
                            )
                        })
                        }

                    </ul>
                </div>
                <div class="card-footer text-center bg-white">
                    <NavLink to="/doctors" className="text-muted">View all Doctors</NavLink>
                </div>
            </div>
        </div>
    </Box>
  );
}
```

7.2 BackEnd

SignUp.js

```
const User = require("../../models/user");
const Doctor = require("../../models/doctor");
const Patient = require("../../models/patient");
const crypto = require('crypto');
const nodemailer = require('nodemailer');
const isUserValid = (newUser) => {
  const errorList = [];
  const nameRegex = /^[a-zA-Z]+(([',. -][a-zA-Z ])?[a-zA-Z]*)*$/;
```

```
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    const passwordRegex = /^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$/;
    if (!newUser.firstName) {
        errorList.push('Please enter first name');
    } else if (!nameRegex.test(newUser.firstName)) {
        errorList.push('First name is invalid');
    }
    if (!newUser.lastName) {
        errorList.push('Please enter last name');
    } else if (!nameRegex.test(newUser.lastName)) {
        errorList.push('Last name is invalid');
    }
    if (!newUser.email) {
        errorList.push("Please enter email");
    } else if (!emailRegex.test(newUser.email)) {
        errorList.push("Invalid email format");
    }
    if (!newUser.password) {
        errorList.push("Please enter password");
    }
    if (!newUser.confirmPassword) {
        errorList.push("Please re-enter password in Confirm Password field");
    }
    if (!newUser.userType) {
        errorList.push("Please enter User Type");
    }
    if (newUser.password !== newUser.confirmPassword) {
        errorList.push("Password and Confirm Password did not match");
    }
    if (errorList.length > 0) {
        return { status: false, errors: errorList };
    } else {
        return { status: true };
    }
};
const saveVerificationToken = async (userId, verificationToken) => {
    await User.findOneAndUpdate({ _id: userId }, { "verificationToken": verificationToken });
    return;
}
const generateVerificationToken = () => {
    const token = crypto.randomBytes(64).toString('hex');
    const expires = Date.now() + 3 * 60 * 60 * 1000;
    let verificationToken = {
        "token": token,
        "expires": expires
    };
    return verificationToken;
};
const sendVerificationEmail = async (email, token) => {
    const transporter = nodemailer.createTransport({
        service: 'gmail',
        auth: {
            user: process.env.GMAIL_USER,
            pass: process.env.GMAIL_PASS
        }
    });
    const mailOptions = {
        from: process.env.GMAIL_USER,
        to: email,
        subject: 'Verify your email address',
        text: `Please click the following link to verify your email address: http://localhost:3001/verify/${token}`,
        html: `<p>Please click this link to verify your account:</p> <a href="http://localhost:3001/verify/${token}">Verify</a>`,
    };
    let resp = await transporter.sendMail(mailOptions);
    return resp;
};
module.exports = (req, res) => {
    const newUser = req.body;
    const userValidStatus = isUserValid(newUser);
    if (!userValidStatus.status) {
```

```
      res.json({ message: "error", errors: userValidStatus.errors });
    } else {
      User.create(
        {
          email: newUser.email,
          username: newUser.email,
          firstName: newUser.firstName,
          lastName: newUser.lastName,
          password: newUser.password,
          userType: newUser.userType,
        },
        (error, userDetails) => {
          if (error) {
            res.json({ message: "error", errors: [error.message] });
          } else {
            let verificationToken = generateVerificationToken()
            saveVerificationToken(userDetails._id, verificationToken);

            if (newUser.userType === "Doctor") {
              Doctor.create(
                {
                  userId: userDetails._id,
                  firstName: newUser.firstName,
                  lastName: newUser.lastName,
                  email: newUser.email,
                  username: newUser.email
                },
                (error2, doctorDetails) => {
                  if (error2) {
                    User.deleteOne({ _id: userDetails });
                    res.json({ message: "error", errors: [error2.message] });
                  } else {
                    let resp = sendVerificationEmail(userDetails.email, verificationToken.token);
                    res.json({ message: "success" });
                  }
                }
              );
            }
            if (newUser.userType === "Patient") {
              Patient.create(
                {
                  userId: userDetails._id,
                  firstName: newUser.firstName,
                  lastName: newUser.lastName,
                  email: newUser.email,
                  username: newUser.email
                },
                (error2, patientDetails) => {
                  if (error2) {
                    User.deleteOne({ _id: userDetails });
                    res.json({ message: "error", errors: [error2.message] });
                  } else {
                    let resp = sendVerificationEmail(userDetails.email, verificationToken.token);
                    res.json({ message: "success" });
                  }
                }
              );
            }
          }
        }
      );
    }
};
```

REFERENCES

[1] Odeh, A., Abdelhadi, R., & Odeh, H. (2019, December). Medical patient appointments management using smart software system in UAE. In 2019 International Arab Conference on Information Technology (ACIT) (pp. 97-101). IEEE.

[2] Wang, M., Le, Y., & Yu, C. H. (2023, December). MedCare4Home: A Mobile Medical System for Families. In 2023 IEEE International Conference on E-health Networking, Application & Services (Healthcom) (pp. 259-264). IEEE.

[3] Enciso, L., Castro, J., & Zelaya-Policarpo, E. (2018). Smart Health: Mobile Application for Booking Medical Appointments. In WEBIST (pp. 438-445).

[4] Aktepe*, A., Turker, A. K., & Ersoz, S. (2015). Internet based intelligent hospital appointment system. Intelligent Automation & Soft Computing, 21(2), 135-146.

[5] Qabajeh, M. M., Mousa, S., Saleh, H. A., & Hasan, J. A. (2021, December). Design and implementation of mobile-based application for appointments systems for vaccinations in medical centers. In 2021 31st International Conference on Computer Theory and Applications (ICCTA) (pp. 54-58). IEEE.

[6] Xiao, Q., Luo, L., Zhao, S. Z., Ran, X. B., & Feng, Y. B. (2018). Online appointment scheduling for a nuclear medicine department in a Chinese hospital. Computational and mathematical methods in medicine, 2018(1), 5148215.

[7] Dai, J., Geng, N., & Xie, X. (2021). Dynamic advance scheduling of outpatient appointments in a moving booking window. European Journal of Operational Research, 292(2), 622-632.

[8] Habibi, M. R. M., Mohammadabadi, F., Tabesh, H., Vakili-Arki, H., Abu-Hanna, A., & Eslami, S. (2019). Effect of an online appointment scheduling system on evaluation metrics of outpatient scheduling system: a before-after multicenterstudy. Journal of medical systems, 43, 1-9.

[9] Valenzuela-Núñez, C., Latorre-Núñez, G., & Espinosa, F. T. (2024). Smart Medical Appointment Scheduling: Optimization, Machine Learning and Overbooking to Enhance Resource Utilization. IEEE Access.

[10] Namakshenas, M., Mazdeh, M. M., Braaksma, A., & Heydari, M. (2023). Appointment scheduling for medical diagnostic centers considering time-sensitive pharmaceuticals: A dynamic robust optimization approach. European journal of operational research, 305(3), 1018-1031.